

Ministerul Educaţiei şi Cercetării
Universitatea "OVIDIUS" Constanţa
Facultatea de Matematică şi Informatică
Specializarea Informatică

CraftManager - Aplicație web pentru managementul atelierelor creative

Lucrare de licenţă

Coordonator ştiinţific:
Conf.univ.dr. Șerban Cristina

Absolvent:
Chiru Silviana

Constanţa
2026

Cuprins

Cuprins i

Lista Figurilor v

Abstract 1

Abstract 1

1 Introducere 3

1.1 Context și motivație 3

1.2 Obiectivele aplicației 4

1.3 Cerințe funcționale 5

1.3.1 Cerințe funcționale pentru vizitatori 5

1.3.2 Cerințe funcționale pentru participanți 5

1.3.3 Cerințe funcționale pentru instructori 6

1.3.4 Cerințe funcționale pentru administrator 6

1.4 Cerințe non-funcționale 7

1.5 Structura lucrării de licență 8

2 Tehnologii utilizate și modele de proiectare 9

2.1 Python 9

2.2 FastAPI 9
2.3 React 9
2.4 Bootstrap 10
2.5 PostgreSQL 10
2.6 HTML 10
2.7 CSS 10
2.8 JavaScript 10
2.9 JSON 11
2.10 JSON Web Token (JWT) 11
2.11 Ollama și Llama 3 11

3 Proiectarea bazei de date 12

3.1 Diagrama Entitate-Asociere 13
3.2 Modelul relațional al datelor 14
3.3 Definirea tabelelor și a constrângerilor 20
3.3.1 Tabela utilizator 20
3.3.2 Tabela atelier 21
3.3.3 Tabela inregistrare 22
3.3.4 Tabela utilizare 22
3.3.5 Tabela recenzie 23

4 Proiectarea aplicației software 24

4.1 Arhitectura generală a aplicației 24
4.2 Model-View-Controller (MVC) 25
4.3 Diagrama cazurilor de utilizare 25
4.4 Descrierea cazurilor de utilizare 27
4.4.1 Cazul de utilizare: Înregistrare utilizator 27
4.4.2 Cazul de utilizare: Autentificare 28

4.4.3 Cazul de utilizare: Filtrare ateliere 29
4.4.4 Cazul de utilizare: Vizualizare detalii atelier 29
4.4.5 Cazul de utilizare: Înscriere la atelier 30
4.4.6 Cazul de utilizare: Anulare înscriere 31
4.4.7 Cazul de utilizare: Gestionare atelier 32
4.4.8 Cazul de utilizare: Asociere materiale 33
4.4.9 Cazul de utilizare: Marcare participare efectivă 34
4.4.10 Cazul de utilizare: Scriere recenzie 35

5 Modulul de Inteligență Artificială 37

5.1 Introducere 37
5.2 Generarea automată a descrierilor pentru workshop-uri 37
5.2.1 Scopul funcționalității 37
5.2.2 Fluxul funcționalității 38
5.2.3 Evaluarea rezultatelor 38
5.2.4 Limitările modulului de generare a descrierilor 40
5.3 Rezumarea automată a feedback-ului pentru workshop-uri 41
5.3.1 Scopul funcționalității 41
5.3.2 Fluxul funcționalității 41
5.3.3 Evaluarea rezultatelor 42
5.3.4 Limitările modulului de rezumare a feedback-ului 43

6 Prezentarea aplicației 44

6.1 Pagina principală 45
6.2 Înregistrarea și autentificarea utilizatorilor 45
6.3 Secțiunea de ateliere 46
6.4 Înscrierile participantului 48
6.5 Dashboard-ul instructorului 48

6.5.1 Crearea atelierelor și generarea de descrieri cu ajutorul AI 48
6.5.2 Evaluările primite și rezumatul AI 50
6.5.3 Raportul financiar 50
6.6 Panoul de administrare 52
6.7 Securitatea aplicației 52

7 Testarea aplicației 53
7.1 Testare funcțională manuală 53
7.2 Testare API cu Postman 54
7.2.1 Înregistrare utilizator - POST /auth/register 54
7.2.2 Autentificare - POST /auth/login 55
7.2.3 Catalog ateliere - GET /ateliere/ 56
7.2.4 Înscriere la atelier - POST /inregistrari/ 57
7.2.5 Acces neautorizat - POST /inregistrari/ fără token 57

8 Concluzii și direcții viitoare 59
8.1 Concluzii 59
8.2 Dificultăți întâmpinate 59
8.3 Direcții viitoare 59

Lista Figurilor

3.1 Diagrama Entitate-Asociere 13

3.2 Baza de date 13

3.3 Constrângerile tabelei utilizator 20

3.4 Constrângerile tabelei atelier 21

3.5 Constrângerile tabelei înregistrare 22

3.6 Constrângerile tabelei utilizare 22

3.7 Constrângerile tabelei recenzie 23

4.1 Diagrama cazurilor de utilizare 26

5.1 Exemplu de descriere generată automat pentru un atelier 39

5.2 Exemplu de răspuns JSON cu mecanism de reîncercare (3 încercări) 40

5.3 Exemplu de rezumat al feedback-ului generat automat 43

5.4 Exemplu de răspuns JSON generat de modulul de analiză a recenziilor 43

6.1 Pagina principală 45

6.2 Formular de înregistrare cu mesaj de eroare pentru email deja înregistrat 46

6.3 Secțiunea de ateliere cu filtrare pe categorii 47

6.4 Filtre avansate: preț, dată, oraș, vârstă, nivel 47

6.5 Modalul de adăugare recenzie pentru un atelier încheiat 48

6.6 Formular de creare atelier în dashboard-ul instructorului 49

6.7 Descriere generată de modulul AI 49

6.8 Evaluările primite și rezumatul generat de AI 50

6.9 Raportul financiar cu detalii per atelier 51

6.10 Panoul de administrare, secțiunea teme și categorii 52

7.1 Postman - POST /auth/register, răspuns 201 Created 54

7.2 Postman - POST /auth/login, răspuns 200 OK cu token JWT 55

7.3 Postman - GET /ateliere/, răspuns 200 OK fără autentificare 56

7.4 Postman - POST /inregistrari/ cu token valid, răspuns 201 Created 57

7.5 Postman - POST /inregistrari/ fără token, răspuns 401 Unauthorized 57

Abstract

Această lucrare își propune să dezvolte o aplicație web pentru gestionarea atelierelor creative, menită să faciliteze procesul de organizare și participare la ateliere. Aceasta este gândită atât din perspectiva instructorilor care doresc să organizeze activități și să atragă cât mai mulți participanți, cât și din perspectiva participanților care caută să participe la activități noi și distractive.

Sistemul urmărește simplificarea procesului de organizare, promovare și gestionare a atelierelor de arte, craft, dezvoltare personală și a altor domenii de relaxare și dezvoltare a creativității, prin intermediul unei platforme web accesibile, moderne și intuitive. În cadrul aplicației a fost implementat un sistem de autentificare bazat pe roluri distincte: participant, instructor și administrator. Instructorii au permisiunea de a crea și de a gestiona ateliere, precum și de a crea și de a edita descrierea, categoria, durata, locația și numărul de participanți. Participantul poate căuta și filtra atelierele disponibile, se poate înscrie și poate oferi recenzii după confirmarea participării.

Aplicația este construită pe o arhitectură de tip client-server, cu un backend RESTful dezvoltat în Python cu FastAPI, o bază de date relațională PostgreSQL și un frontend React cu Bootstrap, care garantează o experiență fluidă și accesibilă pe orice dispozitiv.

Prin funcționalitățile implementate, aplicația oferă o soluție practică pentru administrarea atelierelor creative și contribuie la îmbunătățirea experienței utilizatorilor implicați în organizarea și participarea la astfel de activități.

This paper presents the development of a web application for managing creative workshops, aimed at streamlining the process of organizing and attending creative activities. The platform is designed with two audiences in mind: instructors looking to organize sessions and reach a wider audience, and participants seeking engaging, recreational activities to join.

The system addresses the need for a unified solution to organize, promote, and manage workshops across areas such as arts, crafts, personal development, and other creative and leisure pursuits, all through a modern, accessible, and user-friendly web platform. A role-based authentication system has been implemented, distinguishing between three user types: participant, instructor, and administrator. Instructors can create and manage workshops, including setting and editing details such as description, category, duration, location, and capacity. Participants can browse and filter available workshops, sign up, and leave reviews once their attendance has been confirmed.

The application uses client-server architecture, featuring a RESTful backend built with Python and FastAPI, a PostgreSQL relational database, and a React frontend styled with Bootstrap, together ensuring a responsive and seamless experience across all devices.

Through its core features, the application delivers a practical and efficient solution for managing creative workshops, ultimately improving the experience for everyone involved, whether organizing or attending.

Capitolul 1

Introducere

1.1 Context și motivație

În contextul stilului actual de viață, tot mai mulți oameni caută mijloace prin care să se elibereze de stresul cotidian, să facă lucruri practice, să învețe lucruri noi și să își petreacă timpul liber într-un mod util și relaxant în același timp. Astfel, atelierele creative, de dezvoltare personală, de artă, de tehnologie, de lectură și alte domenii similare pot reprezenta o oportunitate de a ieși din zona de confort și de a învăța lucruri noi fără obligații sau stres.

Astfel de activități pot contribui la stimularea creativității, a abilităților practice și chiar a celor sociale, reprezentând un mediu interactiv în care te poți dezvolta. Activitățile din aceste ateliere pot fi benefice atât pentru adulți de toate vârstele, care pot descoperi noi pasiuni sau se pot relaxa, cât și pentru copii, ajutându-i să se descopere, să se dezvolte și să își îmbunătățească abilitățile sociale.

Pe măsură ce se dezvoltă interesul în această direcție, organizatorii atelierelor se vor confrunta cu diverse provocări în promovarea, organizarea și gestionarea acestora. Organizatorii au nevoie să își gestioneze numărul de participanți, locațiile, tematica și evidența participării.

În acest context, lipsa unui sistem informatic dedicat acestor activități îngreunează gestionarea lor, în special pe măsură ce numărul atelierelor crește și interesul devine mai mare. Astfel, dezvoltarea unei aplicații web pentru gestionarea atelierelor creative poate reprezenta o soluție eficientă pentru organizatori și participanți. Aceasta poate centraliza informațiile, simplifica procesele de înscriere și oferi o organizare mai eficientă a întregului proces.

Cu ajutorul unei aplicații web intuitive, atât organizatorii, cât și participanții beneficiază de un sistem modern care să le îmbunătățească experiența de gestionare și de căutare a atelierelor creative, convingând mai multe persoane să încerce experiențe noi.

1.2 Obiectivele aplicației

Aplicația web CraftManager are ca scop dezvoltarea unei soluții eficiente și moderne pentru gestionarea organizării atelierelor creative. Obiectivele principale ale aplicației sunt:

1. **Crearea unei platforme web accesibile și intuitive** pentru gestionarea atelierelor creative într-un mod simplu și organizat. Interfața aplicației are ca scop să fie ușor de înțeles și de utilizat, astfel încât utilizatorii să poată naviga rapid între funcțiile aplicației și să găsească cu ușurință informațiile necesare.
2. **Implementarea unui sistem de autentificare cu trei roluri distincte:** participant, instructor și administrator. Cu ajutorul acestei separări, oferim perspective diferite asupra aplicației, în funcție de nevoile utilizatorului, asigurând, în plus, securizarea funcțiilor acesteia.
3. **Permisiuni diferite pentru instructori și participanți**: instructorii pot crea, edita și șterge ateliere, iar participanții pot căuta, filtra și se pot înscrie la atelierele disponibile. Utilizatorii au și permisiuni comune, însă este important ca unele funcții să fie diferite.
4. **Integrarea a două module de inteligență artificială** unul pentru generarea automată a descrierilor atelierelor, care sprijină instructorii în redactarea conținutului, și unul pentru rezumarea automată a feedback-ului primit de la participanți care facilitează interpretarea recenziilor.
5. **Afișarea unui catalog practic al atelierelor**, cu filtrare după categorie, preț, dată, oraș și vârstă. Aplicația urmărește să ofere o interfață atractivă și să se încadreze nevoilor fiecărui utilizator.

1.3 Cerințe funcționale

1.3.1 Cerințe funcționale pentru vizitatori

1. Vizualizarea listei de ateliere disponibile, cu informații despre denumire, preț, durată, dată și locație.
2. Căutarea filtrată a atelierelor după categorie, preț minim sau maxim, dată, oraș, nivel și grupă de vârstă, precum și după denumire.
3. Vizualizarea paginii de detalii a unui atelier, descrierea, informațiile despre instructor, data, ora și locația.
4. Alegerea tipului de utilizator (participant sau instructor) la înregistrarea unui cont nou.
5. Accesarea paginilor publice ale aplicației: pagina principală, Despre și Contact.
6. Autentificarea în aplicație pe baza adresei de email și a parolei, folosite la momentul înregistrării contului.

1.3.2 Cerințe funcționale pentru participanți

În plus față de vizitator, participantul are următoarele posibilități:

1. Înscrierea la un atelier disponibil, cu condiția să fie autentificat.
2. Vizualizarea listei propriilor înscrieri, cu statusul fiecăreia (Înscriere, Participare, Anulare).
3. Anularea unei înscrieri active la un atelier.
4. Adăugarea unei recenzii (notă și comentariu) pentru un atelier la care a participat efectiv.
5. Vizualizarea istoricului personal de participare la ateliere.

1.3.3 Cerințe funcționale pentru instructori

În plus față de vizitator, instructorul are următoarele posibilități:

1. Crearea unui atelier nou, cu completarea tuturor detaliilor: denumire, descriere, preț, durată, vârstă recomandată, dată, oră, temă, nivel și locație.
2. Vizualizarea dashboard-ului personal, cu lista tuturor atelierelor create, cu posibilitatea ștergerii permanente sau a editării oricărui câmp.
3. Vizualizarea numărului și a listei participanților înregistrați la fiecare atelier.
4. Marcarea participării efective a participanților la un atelier.
5. Generarea automată a descrierii unui atelier prin intermediul unui modul de inteligență artificială integrat în platformă.
6. Generarea automată a unui rezumat al recenziilor primite pentru un atelier, prin intermediul unui modul de inteligență artificială.

1.3.4 Cerințe funcționale pentru administrator

1. Autentificarea cu un cont dedicat.
2. Vizualizarea listei tuturor utilizatorilor înregistrați.
3. Ștergerea unui cont de utilizator din platformă.
4. Vizualizarea tuturor atelierelor existente pe platformă.
5. Adăugarea și ștergerea de categorii noi.
6. Adăugarea și ștergerea temelor noi asociate unei categorii.
7. Vizualizarea și ștergerea mesajelor trimise prin formularul de contact.

1.4 Cerințe non-funcționale

1. **Performanța.**
   Aplicația trebuie să răspundă la cererile utilizatorului în mai puțin de 2 secunde în condiții normale de utilizare.

2. **Securitate.**
   Parolele utilizatorilor sunt stocate în baza de date sub formă de hash (bcrypt), iar autentificarea se realizează prin token JWT cu timp de expirare configurabil.

3. **Uzabilitate.**
   Interfața trebuie să fie intuitivă, cu un design curat și modern, și accesibilă persoanelor fără experiență. Navigarea între pagini trebuie să fie clară, iar funcționalitățile importante să fie accesibile rapid.

4. **Compatibilitate.**
   Aplicația trebuie să fie complet funcțională, adaptabilă și să aibă un aspect plăcut atât pe desktop, cât și pe dispozitive mobile.

5. **Scalabilitate.**
   Arhitectura aplicației permite adăugarea de noi funcționalități fără a afecta componentele existente. Sistemul trebuie să poată susține un număr mai mare de utilizatori și ateliere pe măsură ce platforma se dezvoltă.

6. **Fiabilitate.**
   Datele introduse sunt validate atât la nivelul frontend-ului, cât și la nivelul backend-ului, prevenind introducerea de date incorecte.

7. **Consistența datelor.**
   Baza de date include constrângeri de integritate (chei străine, constrângeri de unicitate, validări de domeniu) pentru a asigura corectitudinea datelor.

8. **Portabilitate.**
   Aplicația poate fi rulată pe sisteme de operare diferite, cu condiția instalării mediului de execuție și a dependențelor necesare

1.5 Structura lucrării de licență

Capitolul 1 - Introducere. În acest capitol sunt prezentate contextul dezvoltării aplicației și motivația alegerii temei. Oferă o descriere generală a scopului aplicației, a obiectivelor principale și a cerințelor funcționale și nefuncționale ale sistemului, precum și a motivației personale pentru alegerea temei.

Capitolul 2 - Tehnologii utilizate și modele de proiectare. Capitolul prezintă tehnologiile utilizate în dezvoltarea aplicației, atât pentru partea de backend, cât și pentru frontend și baza de date. Sunt descrise framework-urile și instrumentele folosite, precum FastAPI, React, PostgreSQL și Ollama. Totodată, sunt prezentate modelele de proiectare și arhitectura aplicației.

Capitolul 3 - Proiectarea bazei de date. În acest capitol sunt prezentate entitățile sistemului și relațiile dintre acestea. Sunt incluse diagrama entitate-asociere, modelul relațional al bazei de date, restricțiile de integritate și structura tabelelor utilizate pentru stocarea informațiilor despre utilizatori, ateliere, înscrieri și recenzii.

Capitolul 4 - Proiectarea aplicației software. Capitolul descrie proiectarea aplicației din punctul de vedere funcțional. Sunt prezentate cazurile de utilizare, actorii sistemului, diagramele UML și funcționalitățile principale implementate pentru participanți și pentru instructori. De asemenea, este explicat fluxul principal al aplicației și modul de interacțiune dintre componente.

Capitolul 5 - Modulul de inteligență artificială. Acest capitol detaliază funcționalitatea de generare automată a descrierilor pentru ateliere, bazată pe modele de tip Large Language Model. Sunt explicate procesul de creare a prompturilor, validarea automată a răspunsurilor și metoda de evaluare a rezultatelor.

Capitolul 6 - Implementarea și prezentarea aplicației. În acest capitol sunt prezentate componentele principale ale aplicației și modul lor de funcționare. Sunt incluse exemple din interfața grafică, explicații despre fluxurile utilizatorilor și fragmente de cod relevante pentru funcționalitățile esențiale ale sistemului.

Capitolul 7 - Concluzii și direcții viitoare. Ultimul capitol prezintă concluziile studiului, principalele rezultate și experiența dobândită în dezvoltarea aplicației. De asemenea, explorează posibile direcții de extindere și îmbunătățire a sistemului în viitor.

Capitolul 2

Tehnologii utilizate și modele de proiectare

2.1 Python

Limbajul principal utilizat în dezvoltarea backend-ului este Python [**?**]. Am ales utilizarea acestui limbaj datorită sintaxei clare, ușor de înțeles, varietății de biblioteci disponibile pentru dezvoltarea aplicațiilor web și pentru integrarea funcționalităților de inteligență artificială. Este un limbaj frecvent utilizat în dezvoltarea aplicațiilor datorită flexibilității și ușurinței în implementare.

2.2 FastAPI

Pentru dezvoltarea backend-ului, a fost utilizat framework-ul FastAPI [**?**]. Acesta este un framework modern pentru dezvoltarea de API-uri REST în Python. Oferă performanțe bune și o structură clară care facilitează dezvoltarea aplicațiilor web. Ajută, de asemenea, la implementarea operațiilor CRUD și la comunicarea rapidă între frontend și backend.

2.3 React

Frontend-ul aplicației a fost dezvoltat cu ajutorul bibliotecii React [**?**]. Acesta permite construirea de interfețe web care oferă o experiență interactivă și dinamică utilizatorilor. Utilizarea componentelor reutilizabile ajută la o organizare mai clară a codului. Este una dintre cele mai utilizate tehnologii pentru dezvoltarea aplicațiilor web de tip Single Page Application.

2.4 Bootstrap

Pentru designul aplicației a fost utilizat framework-ul Bootstrap [**?**]. Acesta oferă elemente predefinite și un mod flexibil de organizare a elementelor în pagină, contribuind la dezvoltarea unei interfețe curate și simplu de utilizat, adaptabilă pe orice dispozitiv. Ajută la stilizarea aplicațiilor web fără a mai fi nevoie de mult CSS.

2.5 PostgreSQL

Pentru stocarea și gestionarea datelor aplicației a fost utilizat PostgreSQL [**?**]. Acesta este un sistem relațional open-source cunoscut pentru performanță, stabilitate și suportul pentru operații complexe aplicate asupra datelor. Utilizarea unei baze de date relaționale permite definirea clară a relațiilor dintre entități și menținerea integrității datelor prin utilizarea cheilor primare și a cheilor externe.

2.6 HTML

HTML reprezintă limbajul standard utilizat pentru construirea și structurarea paginilor web. Acesta permite organizarea elementelor vizuale din interfața unei aplicații, precum titluri, texte, formulare, imagini, butoane sau tabele. În cadrul aplicației, HTML este utilizat împreună cu React pentru realizarea interfeței grafice.

2.7 CSS

CSS este limbajul utilizat pentru stilizarea și personalizarea elementelor HTML. Cu ajutorul CSS pot fi definite culorile, dimensiunile, poziționarea elementelor și aspectul general al aplicației. În aplicație, CSS este utilizat pentru realizarea unei interfețe moderne și pentru stilizarea paginilor, împreună cu framework-ul Bootstrap.

2.8 JavaScript

JavaScript este unul dintre cele mai utilizate limbaje de programare pentru dezvoltarea aplicațiilor web. Acesta contribuie la realizarea unei experiențe interactive și rapide pentru utilizatori, permițând actualizarea informațiilor fără a reîncărca complet pagina. În cadrul aplicației, este utilizat prin intermediul bibliotecii React.

2.9 JSON

JSON este un format de text utilizat pentru schimbul și stocarea datelor între aplicații. Formatul este ușor de citit și de utilizat atât de către oameni, cât și de către aplicații software. În aplicație, JSON este utilizat pentru transmiterea datelor între frontend și backend, precum și pentru procesarea răspunsurilor generate de modelul AI.

2.10 JSON Web Token (JWT)

JSON Web Token (JWT) [**?**] reprezintă un standard utilizat pentru autentificarea și autorizarea utilizatorilor în aplicațiile web. JWT permite transmiterea semnată a unor informații între client și server; confidențialitatea este asigurată doar dacă transmiterea se face prin HTTPS. În aplicație, JWT este utilizat pentru autentificarea utilizatorilor și pentru gestionarea sesiunilor active.

2.11 Ollama și Llama 3

Ollama este o platformă open-source [**?**] care permite rularea locală a modelelor de tip Large Language Model (LLM). Aceasta facilitează integrarea modelelor de limbaj în aplicații fără a necesita conectarea la servicii externe. Comunicarea cu Ollama se realizează prin intermediul unui API REST local, apelat din backend-ul FastAPI.

Capitolul 3

Proiectarea bazei de date

Baza de date reprezintă un element esențial al aplicației, având rolul de a stoca și de a gestiona toate informațiile necesare. Proiectarea acesteia a fost realizată folosind modelul relațional, care permite organizarea eficientă a datelor și definirea corectă a relațiilor dintre entități. Structura bazei de date a fost construită urmând cu strictețe toți pașii, astfel încât să asigurăm integritatea și consistența datelor prin utilizarea cheilor primare, a cheilor externe și a constrângerilor.

Capitolul urmărește prezentarea diagramei entitate-asociere, a principalelor entități și a relațiilor dintre acestea, a modelului relațional al bazei de date, dar și a constrângerilor, toate contribuind la funcționarea corectă a aplicației.

3.1 Diagrama Entitate-Asociere

Modelul Entități-Asociații urmărește o reprezentare grafică sugestivă, care modelează universul problemei ca o mulțime de entități și relații dintre acestea. Diagrama DEA ajută la proiectarea corectă a bazei de date [**?**].

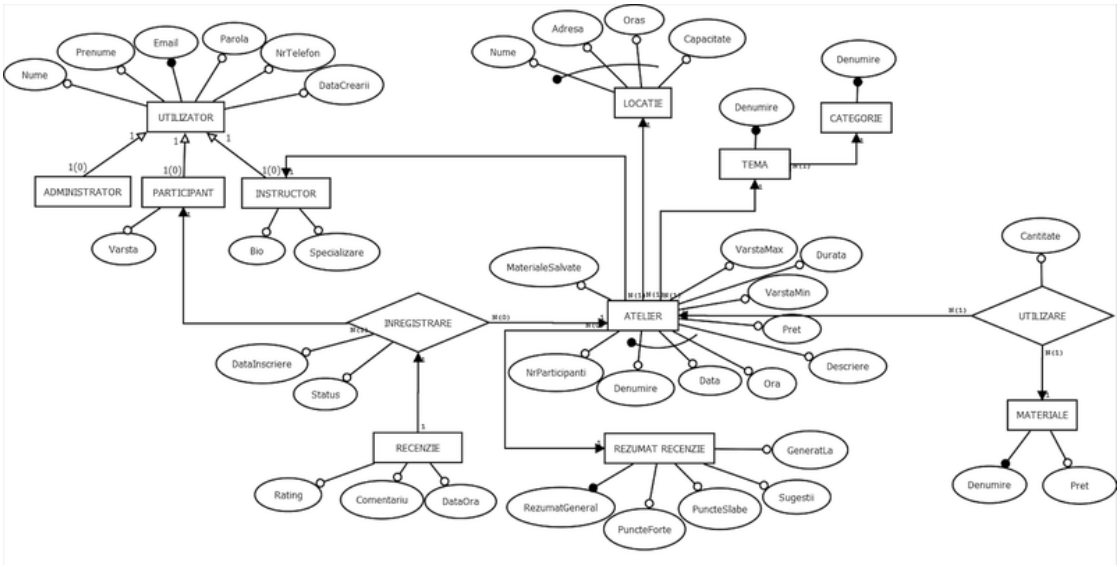


Figura 3.1: Diagrama Entitate-Asociere

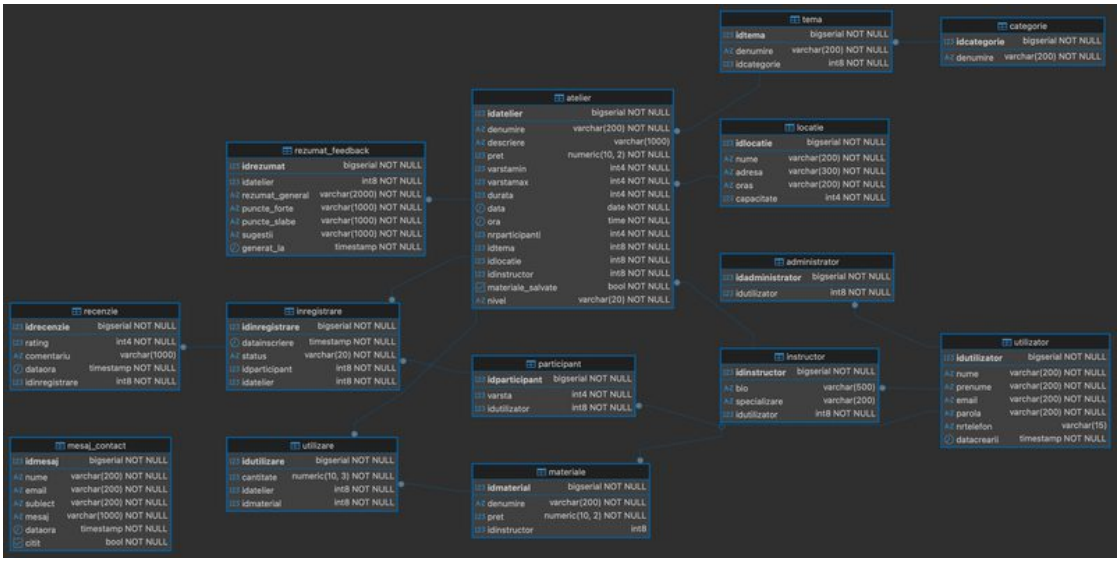


Figura 3.2: Baza de date

3.2 Modelul relațional al datelor

1. UTILIZATOR(<u>IdUtilizator</u>, Nume, Prenume, Email, Parola, NrTelefon, DataCrearii)

Chei semantice: Email

Constrângeri de domeniu:

$\Rightarrow$ Dom(Nume) = CHAR(200)
$\Rightarrow$ Dom(Prenume) = CHAR(200)
$\Rightarrow$ Dom(Email) = CHAR(200)
$\Rightarrow$ Dom(Parola) = CHAR(200) | lungime ≥ 8
$\Rightarrow$ Dom(NrTelefon) = $\{$CHAR(15) | $10 \leq$ length(s) $\leq 15\}$
$\Rightarrow$ Dom(DataCrearii) = $\{d \in$ Date | 01.01.2000 $\leq$ d $\leq$ 31.12.2100$\}$

Constrângeri de existență: Nume, Prenume, Email, Parola, NrTelefon
Constrângeri de integritate referențială: -

2. PARTICIPANT(<u>IdUtilizator</u>, Varsta)

Cheie primară/semantică: IdUtilizator

Constrângeri de domeniu:

$\Rightarrow$ Dom(Varsta) = $\{n \in \mathbb{N} \mid 1 \leq n \leq 120\}$

Constrângeri de existență: IdUtilizator, Varsta
Constrângeri de integritate referențială:
IdUtilizator $\subseteq$ UTILIZATOR.IdUtilizator

3. INSTRUCTOR(<u>IdUtilizator</u>, Bio, Specializare)

Cheie primară/semantică: IdUtilizator

Constrângeri de domeniu:

$\Rightarrow$ Dom(Bio) = CHAR(500)
$\Rightarrow$ Dom(Specializare) = CHAR(50)

Constrângeri de existență: IdUtilizator, Bio, Specializare
Constrângeri de integritate referențială:
IdUtilizator $\subseteq$ UTILIZATOR.IdUtilizator

4. LOCATIE(<u>IdLocatie</u>, Nume, Adresa, Oras, Capacitate)

Chei semantice: Nume $*$ Adresa $*$ Oras

Constrângeri de domeniu:

$\Rightarrow$ Dom(Nume) = CHAR(200)
$\Rightarrow$ Dom(Adresa) = CHAR(200)
$\Rightarrow$ Dom(Oras) = CHAR(50)
$\Rightarrow$ Dom(Capacitate) = $\{n \in \mathbb{N} \mid 1 \leq n \leq 5000\}$

Constrângeri de existență: Nume, Adresa, Oras, Capacitate
Constrângeri de integritate referențială: -

5. CATEGORIE(<u>IdCategorie</u>, Denumire)

Chei semantice: Denumire

Constrângeri de domeniu:

$\Rightarrow$ Dom(Denumire) = CHAR(50)

Constrângeri de existență: Denumire
Constrângeri de integritate referențială: -

6. TEMA(<u>IdTema</u>, Denumire, IdCategorie)

Chei semantice: Denumire $*$ IdCategorie

Constrângeri de domeniu:

$\Rightarrow$ Dom(Denumire) = CHAR(50)

Constrângeri de existență: Denumire, IdCategorie
Constrângeri de integritate referențială:
IdCategorie $\subseteq$ CATEGORIE.IdCategorie

7. ATELIER(<u>IdAtelier</u>, Denumire, Descriere, Pret, VarstaMin, VarstaMax, Durata, Data, Ora, MaterialeSalvate, Nivel, NrParticipanti, IdTema, IdInstructor, IdLocatie)

Chei semantice: Denumire $*$ Data $*$ Ora $*$ IdLocatie

Constrângeri de domeniu:

$\Rightarrow$ Dom(Denumire) = CHAR(100)
$\Rightarrow$ Dom(Descriere) = CHAR(1000)
$\Rightarrow$ Dom(Pret) = $\{n \in \mathbb{N} \mid 1 \leq n \leq 50000\}$
$\Rightarrow$ Dom(VarstaMin) = $\{n \in \mathbb{N} \mid 1 \leq n \leq 120\}$
$\Rightarrow$ Dom(VarstaMax) = $\{n \in \mathbb{N} \mid 1 \leq n \leq 120\}$
$\Rightarrow$ Dom(Durata) = $\{n \in \mathbb{N} \mid 1 \leq n \leq 1440\}$ (minute)
$\Rightarrow$ Dom(Data) = $\{d \in$ Date $\mid$ 01.01.2000 $\leq$ d $\leq$ 31.12.2100$\}$
$\Rightarrow$ Dom(Ora) = $\{t \in$ Time $\mid$ 00:00 $\leq$ t $\leq$ 23:59$\}$
$\Rightarrow$ Dom(MaterialeSalvate) = Boolean
$\Rightarrow$ Dom(NrParticipanti) = $\{n \in \mathbb{N} \mid 0 \leq n \leq 10000\}$
$\Rightarrow$ Dom(Nivel) = CHAR(20)

Constrângeri de existență: Denumire, IdTema, IdInstructor
Constrângeri de integritate referențială:
IdTema $\subseteq$ TEMA.IdTema
IdInstructor $\subseteq$ INSTRUCTOR.IdInstructor
IdLocatie $\subseteq$ LOCATIE.IdLocatie
Constrângeri de tuplu:
NrParticipanti $\leq$ LOCATIE.Capacitate

8. INREGISTRARE(<u>IdInregistrare</u>, DataInscriere, Status, IdParticipant, IdAtelier)

Chei semantice: IdParticipant $*$ IdAtelier $*$ DataInscriere

Constrângeri de domeniu:

$\Rightarrow$ Dom(DataInscriere) = $\{d \in$ DateTime $\mid$ 01.01.2000 00:00:00 $\leq$ d $\leq$ 31.12.2100 23:59:59$\}$
$\Rightarrow$ Dom(Status) = {„Inscriere”, „Participare”, „Anulare”}

Constrângeri de existență: DataInscriere, Status, IdParticipant, IdAtelier
Constrângeri de integritate referențială:
IdParticipant $\subseteq$ PARTICIPANT.IdParticipant
IdAtelier $\subseteq$ ATELIER.IdAtelier
Constrângeri de tuplu:
DataInscriere $\leq$ ATELIER.Data
DataInscriere $\leq$ PARTICIPANT.DataCreare

9. RECENZIE(<u>IdRecenzie</u>, Rating, Comentariu, DataOra, IdInregistrare)

Chei semantice: IdInregistrare

Constrângeri de domeniu:

$\Rightarrow$ Dom(Rating) = $\{n \in \mathbb{N} \mid 1 \leq n \leq 5\}$
$\Rightarrow$ Dom(Comentariu) = CHAR(1000)
$\Rightarrow$ Dom(DataOra) = $\{d \in$ DateTime $\mid$ 01.01.2000 00:00:00 $\leq$ d $\leq$ 31.12.2100 23:59:59$\}$

Constrângeri de existență: Rating, DataOra, IdInregistrare
Constrângeri de integritate referențială:
IdInregistrare $\subseteq$ INREGISTRARE.IdInregistrare
Constrângeri de tuplu:
DataOra $\geq$ ATELIER.DataOra + Durata

10. MATERIALE(<u>IdMaterial</u>, Denumire, Pret)

Chei semantice: Denumire

Constrângeri de domeniu:

$\Rightarrow$ Dom(Denumire) = CHAR(100)
$\Rightarrow$ Dom(Pret) = $\{n \in \mathbb{N} \mid 1 \leq n \leq 100000\}$

Constrângeri de existență: Denumire, Pret
Constrângeri de integritate referențială:
IdAtelier $\subseteq$ ATELIER.IdAtelier

11. UTILIZARE(<u>IdUtilizare</u>, Cantitate, IdAtelier, IdMaterial)

Chei semantice: IdAtelier $*$ IdMaterial

Constrângeri de domeniu:

$\Rightarrow$ Dom(Cantitate) = $\{n \in \mathbb{R} \mid 0 < n \leq 100000\}$

Constrângeri de existență: Cantitate, IdAtelier, IdMaterial
Constrângeri de integritate referențială:
IdAtelier $\subseteq$ ATELIER.IdAtelier
IdMaterial $\subseteq$ MATERIALE.IdMaterial

12. ADMINISTRATOR(IdAdministrator, IdUtilizator)

Chei semantice: IdUtilizator

Constrângeri de domeniu: -

Constrângeri de existență: IdUtilizator
Constrângeri de integritate referențială:
IdUtilizator $\subseteq$ UTILIZATOR.IdUtilizator

13. MESAJ_CONTACT(IdMesaj, Nume, Email, Subiect, Mesaj, DataOra, Citit)

Chei semantice: Nume, Email

Constrângeri de domeniu:

$\Rightarrow$ Dom(Nume) = CHAR(200)
$\Rightarrow$ Dom(Email) = CHAR(200)
$\Rightarrow$ Dom(Subiect) = CHAR(200)
$\Rightarrow$ Dom(Mesaj) = CHAR(1000)
$\Rightarrow$ Dom(DataOra) = $\{d \in$ DateTime $\mid$ 01.01.2000 00:00:00 $\leq$ d $\leq$ 31.12.2100 23:59:59$\}$
$\Rightarrow$ Dom(Citit) = Boolean

Constrângeri de existență: Nume, Email, Subiect, Mesaj, DataOra
Constrângeri de integritate referențială: -

14. REZUMAT_FEEDBACK(IdRezumat, IdAtelier, RezumatGeneral, PuncteForte, PuncteSlabe, Sugestii, GeneratLa)

Chei semantice: IdAtelier $*$ RezumatGeneral

Constrângeri de domeniu:

$\Rightarrow$ Dom(RezumatGeneral) = CHAR(2000)
$\Rightarrow$ Dom(PuncteForte) = CHAR(1000)
$\Rightarrow$ Dom(PuncteSlabe) = CHAR(1000)
$\Rightarrow$ Dom(Sugestii) = CHAR(1000)
$\Rightarrow$ Dom(GeneratLa) = $\{d \in$ DateTime $\mid$ 01.01.2000 00:00:00 $\leq$ d $\leq$ 31.12.2100 23:59:59$\}$

Constrângeri de existență: IdAtelier, RezumatGeneral, PuncteForte, PucteSlabe, Sugestii, GeneratLa
Constrângeri de integritate referențială:
IdAtelier $\subseteq$ ATELIER.IdAtelier

Restricții

1. Nu pot exista doi utilizatori cu același Email.
2. Un utilizator poate fi fie participant, fie instructor.
3. Specializarea UTILIZATOR → PARTICIPANT / INSTRUCTOR este disjunctă: același IdUtilizator nu poate apărea în ambele sub-entități.
4. Nu pot exista două categorii cu aceeași Denumire.
5. Nu pot exista două teme cu aceeași denumire în aceeași categorie.
6. Nu pot exista două locații cu aceeași denumire, la aceeași adresă.
7. Nu pot exista două ateliere cu aceeași denumire care să se desfășoare la aceeași dată și în aceeași locație.
8. Un participant la un atelier se poate înscrie, se poate retrage, apoi se poate înscrie iar la același atelier (dar nu în aceeași zi).
9. Un participant înregistrat la un atelier poate oferi o singură recenzie pentru acel atelier (unicitate pe RECENZIE.IdInregistrare).
10. Nu pot exista două materiale cu aceeași denumire.
11. Fiecare material poate apărea cel mult o dată pentru același atelier (în tabela UTILIZARE).
12. Mesajele de contact nu sunt asociate unui utilizator și pot fi trimise anonim.
13. Un rezumat de feedback poate exista doar dacă există cel puțin o recenzie cu comentariu pentru atelierul respectiv.

Buclă comutativă: INREGISTRARE → ATELIER → INSTRUCTOR → UTILIZATOR și INREGISTRARE → PARTICIPANT → UTILIZATOR

Rezolvare: Un utilizator nu poate fi participant și instructor în același atelier.

3.3 Definirea tabelelor și a constrângerilor

În această secțiune sunt prezentate constrângerile definite la nivelul tabelelor din baza de date PostgreSQL [**?**].

3.3.1 Tabela utilizator

Tabela utilizator stochează informații generale despre utilizatorii aplicației, fie ei instructori sau participanți. Pentru asigurarea corectitudinii informațiilor înregistrate, pentru a preveni duplicatele sau informațiile invalide, au fost definite constrângeri. Acestea au rolul de a identifica unic fiecare utilizator, de a preveni înregistrarea mai multor conturi cu același email și de a impune lungimi pentru parolă și pentru numărul de telefon.

```
CREATE TABLE utilizator(
idutilizator BIGSERIAL PRIMARY KEY,
nume VARCHAR(200) NOT NULL,
prenume VARCHAR(200) NOT NULL,
email VARCHAR(200) NOT NULL,
parola VARCHAR(200) NOT NULL,
nrtelefon VARCHAR(15),
datacrearii TIMESTAMP NOT NULL DEFAULT NOW(),

CONSTRAINT uq_utilizator_email UNIQUE(email),
CONSTRAINT ck_utilizator_email_format CHECK(position('@' in email)>1),
CONSTRAINT ck_utilizator_parola_len CHECK(char_length(parola)>=8),
CONSTRAINT ck_utilizator_nrtelefon_len CHECK(nrtelefon IS NULL OR(char_length(nrtelefon) BETWEEN 10 AND 15))
);
```

Figura 3.3: Constrângerile tabelei utilizator

3.3.2 Tabela atelier

Tabela atelier reprezintă centrul principal al aplicației. Aceasta conține toate informațiile necesare pentru organizarea atelierelor. Au fost definite constrângeri pentru a preveni introducerea de valori negative, pentru validarea intervalului de vârstă și pentru a limita durata atelierului.

```
CREATE TABLE atelier(
  idatelier BIGSERIAL PRIMARY KEY,
  denumire VARCHAR(200) NOT NULL,
  descriere VARCHAR(1000),
  pret NUMERIC(10,2) NOT NULL,
  varstamin INT NOT NULL,
  varstamax INT NOT NULL,
  durata INT NOT NULL,
  data DATE NOT NULL,
  ora TIME NOT NULL,
  nrparticipanti INT NOT NULL DEFAULT 0,
  idtema BIGINT NOT NULL,
  idlocatie BIGINT NOT NULL,
  idinstructor BIGINT NOT NULL,

  CONSTRAINT fk_atelier_tema FOREIGN KEY(idtema)
  REFERENCES tema(idtema) ON DELETE RESTRICT,

  CONSTRAINT fk_atelier_locatie FOREIGN KEY(idlocatie)
  REFERENCES locatie(idlocatie) ON DELETE RESTRICT,

  CONSTRAINT fk_atelier_instructor FOREIGN KEY(idinstructor)
  REFERENCES instructor(idinstructor) ON DELETE RESTRICT,

  CONSTRAINT ck_atelier_pret CHECK(pret>=0),

  CONSTRAINT ck_atelier_varste CHECK(
  varstamin BETWEEN 1 AND 120 AND
  varstamax BETWEEN 1 AND 120 AND
  varstamin <= varstamax
  ),

  CONSTRAINT ck_atelier_durata CHECK(durata BETWEEN 15 AND 1440),
  CONSTRAINT ck_atelier_nrparticipanti CHECK(nrparticipanti>=0),

  CONSTRAINT uq_atelier_semantic UNIQUE(denumire,data,ora,idlocatie)
  );
```

Figura 3.4: Constrângerile tabelei atelier

3.3.3 Tabela inregistrare

Tabela inregistrare este o tabelă asociativă care face legătura între participanți și ateliere, permițând înscrierea mai multor participanți la un atelier și participarea unui participant la mai multe ateliere. Constrângerile restricționează un participant să nu se înscrie la același atelier de două ori și impun ca statusul înscrierii să aibă doar valori prestabilite. Tabela permite gestionarea istoricului de participare la ateliere.

```
CREATE TABLE inregistrare(
   idinregistrare BIGSERIAL PRIMARY KEY,
   datainscriere TIMESTAMP NOT NULL DEFAULT NOW(),
   status VARCHAR(20) NOT NULL,
   idparticipant BIGINT NOT NULL,
   idatelier BIGINT NOT NULL,

   CONSTRAINT fk_inregistrare_participant FOREIGN KEY(idparticipant)
   REFERENCES participant(idparticipant) ON DELETE CASCADE,

   CONSTRAINT fk_inregistrare_atelier FOREIGN KEY(idatelier)
   REFERENCES atelier(idatelier) ON DELETE CASCADE,

   CONSTRAINT uq_inregistrare UNIQUE(idparticipant,idatelier),

   CONSTRAINT ck_inregistrare_status CHECK(status IN('Inscriere','Participare','Anulare'))
);
```

Figura 3.5: Constrângerile tabelei înregistrare

3.3.4 Tabela utilizare

Tabela utilizare este o tabelă asociativă care face legătura între ateliere și materiale. Aceasta permite asocierea mai multor materiale la un atelier și asocierea unui material la mai multe ateliere. Tabela permite gestionarea resurselor necesare pentru un atelier. Aceeași combinație atelier-material nu poate exista de două ori, iar cantitatea trebuie să fie pozitivă.

```
CREATE TABLE utilizare(
  idutilizare BIGSERIAL PRIMARY KEY,
  cantitate NUMERIC(10,3) NOT NULL,
  idatelier BIGINT NOT NULL,
  idmaterial BIGINT NOT NULL,

  CONSTRAINT fk_utilizare_atelier FOREIGN KEY(idatelier)
  REFERENCES atelier(idatelier) ON DELETE CASCADE,

  CONSTRAINT fk_utilizare_material FOREIGN KEY(idmaterial)
  REFERENCES materiale(idmaterial) ON DELETE RESTRICT,

  CONSTRAINT uq_utilizare UNIQUE(idatelier,idmaterial),
  CONSTRAINT ck_utilizare_cantitate CHECK(cantitate>0)
  );
```

Figura 3.6: Constrângerile tabelei utilizare

3.3.5 Tabela recenzie

Tabela de recenzie permite participanților efectivi la ateliere să ofere recenzii, note și comentarii pentru atelierul la care au participat. Recenziile ajută instructorii să atragă participanți noi și îi ajută pe participanți să își facă o idee despre atelierul la care urmează să participe. Comentariile oferă o perspectivă asupra experienței participantului. Nota poate avea valori strict între 1 și 5.

```
CREATE TABLE recenzie(
idrecenzie BIGSERIAL PRIMARY KEY,
rating INT NOT NULL,
comentariu VARCHAR(1000),
dataora TIMESTAMP NOT NULL DEFAULT NOW(),
idinregistrare BIGINT NOT NULL,

CONSTRAINT fk_recenzie_inregistrare FOREIGN KEY(idinregistrare)
REFERENCES inregistrare(idinregistrare) ON DELETE CASCADE,

CONSTRAINT uq_recenzie_inregistrare UNIQUE(idinregistrare),
CONSTRAINT ck_recenzie_rating CHECK(rating BETWEEN 1 AND 5)
);
```

Figura 3.7: Constrângerile tabelei recenzie

Capitolul 4

Proiectarea aplicației software

4.1 Arhitectura generală a aplicației

În dezvoltarea aplicației „CraftManager” a fost folosită o arhitectură de tip server-client [**?**]. Acest tip de arhitectură descrie un model de calcul în care unul sau mai multe computere client solicită resurse de la un computer server. Acest model poate permite mai multor utilizatori să acceseze și să utilizeze simultan aceeași resursă. Modelul arhitectural împarte sistemul în două componente principale: client și server, care comunică între ele prin intermediul cererilor HTTP.

Clientul reprezintă partea vizibilă utilizatorului, interfața grafică a aplicației. Datele introduse de utilizator sunt trimise către server prin intermediul cererilor HTTP, iar răspunsurile primite sunt afișate în interfață într-un mod dinamic.

Serverul reprezintă partea pe care utilizatorul nu o poate vedea, componenta responsabilă de procesarea datelor și de gestionarea logicii aplicației. Comunicarea dintre client și server se realizează prin intermediul unui API REST, utilizând metode HTTP precum GET, POST, PUT, PATCH și DELETE [**?**].

- ▷ Metoda GET este utilizată pentru obținerea informațiilor de la server, fără a modifica datele existente.
- ▷ Metoda POST este utilizată pentru a trimite date către server și pentru a crea noi resurse în baza de date.
- ▷ Metoda PUT este utilizată de regulă pentru actualizarea completă a unei resurse existente; în unele implementări poate crea resursa dacă aceasta nu există.
- ▷ Metoda PATCH este utilizată pentru modificarea parțială a unei resurse.
- ▷ Metoda DELETE este utilizată pentru ștergerea resurselor din sistem.

4.2 Model-View-Controller (MVC)

Pentru dezvoltarea aplicației a fost aplicat principiul separării responsabilităților, inspirat din modelul arhitectural Model-View-Controller (MVC) [**?**]. Frontend-ul React joacă rolul de View, prezentând datele utilizatorului, backend-ul FastAPI gestionează logica aplicației și expune endpoint-uri API (Controller), iar modelele SQLAlchemy reprezintă componenta Model, responsabilă de structura și accesul la date. Această separare contribuie la organizarea mai eficientă a codului și facilitează dezvoltarea, întreținerea și extinderea viitoare a aplicației.

Componenta Model este responsabilă de gestionarea datelor și a logicii aplicației. Această componentă comunică direct cu baza de date și conține structurile utilizate pentru stocarea informațiilor necesare funcționării corecte a aplicației.

Componenta View reprezintă interfața grafică afișată utilizatorului. Aceasta are rolul de a prezenta informațiile într-un mod accesibil și intuitiv pentru orice utilizator.

Componenta Controller realizează legătura dintre interfață și logica aplicației. Aceasta primește cererile utilizatorului, procesează datele și transmite răspunsurile către View sau către Model.

4.3 Diagrama cazurilor de utilizare

Diagrama cazurilor de utilizare [**?**] este o diagramă UML utilizată pentru a reprezenta interacțiunea dintre utilizatori și sistem. Aceasta oferă o imagine generală asupra funcționalităților aplicației și asupra modului în care diferiți tipuri de utilizatori interacționează cu sistemul. Diagrama este utilă în etapa de analiză și proiectare deoarece permite identificarea clară a cerințelor funcționale și a responsabilităților fiecărui actor din aplicație.

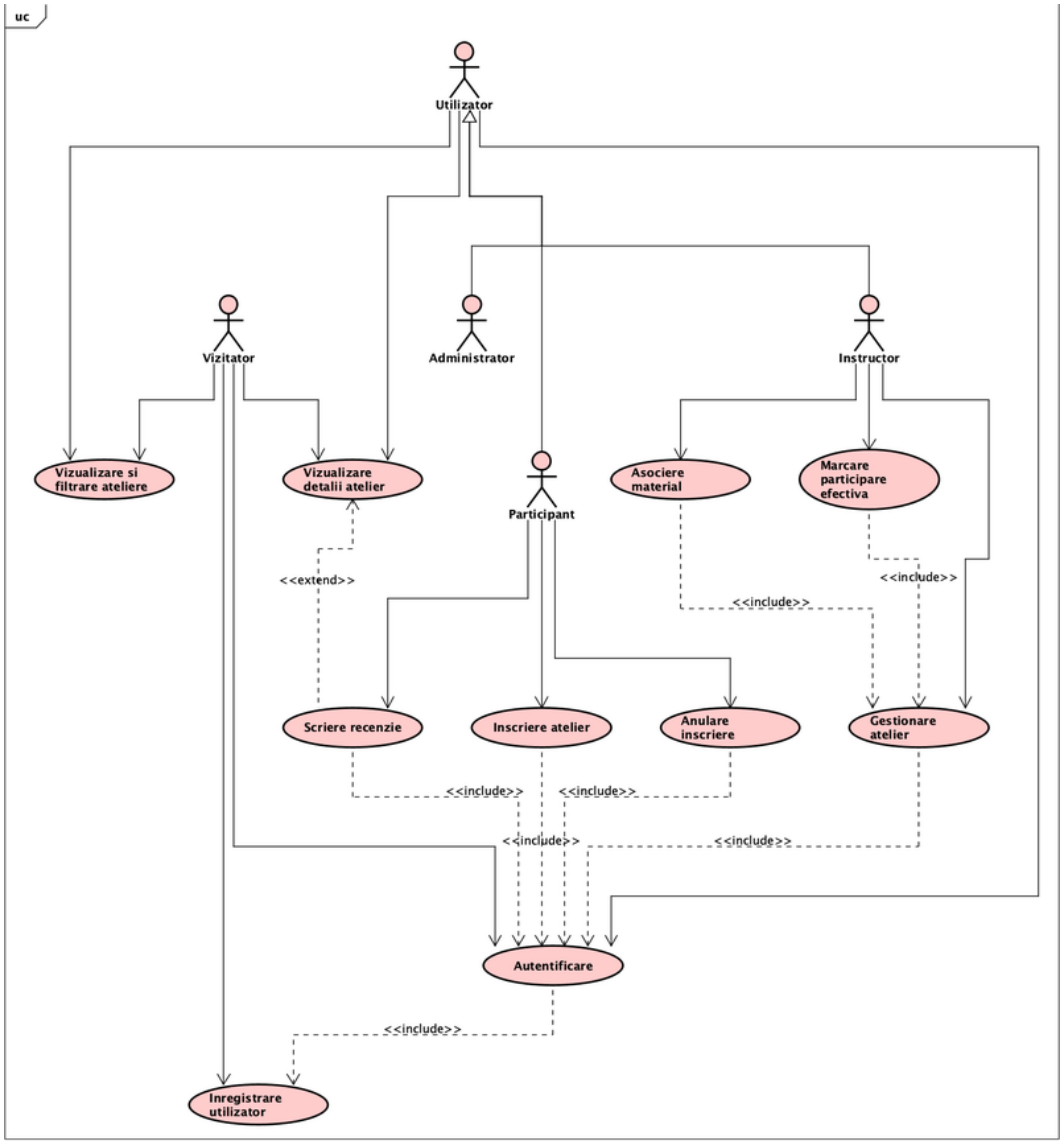


Figura 4.1: Diagrama cazurilor de utilizare

4.4 Descrierea cazurilor de utilizare

În această secțiune sunt descrise în detaliu zece cazuri de utilizare pentru aplicația Craft-Manager [**?**].

4.4.1 Cazul de utilizare: Înregistrare utilizator

Nume: Înregistrare utilizator
Descriere: Descrie comportamentul sistemului și procesul de comunicare al unui utilizator, fie participant, fie instructor, care dorește să creeze un cont nou în aplicație.
Actor software: Vizitator
Eveniment declanșator: Vizitator cere crearea unui cont.
Precondiții: Sistemul funcționează corect.
Postcondiții: Sistemul a memorat contul și a creat un profil corespunzător, fie de instructor, fie de participant.
Referințe încrucișate: Autentificare

Flux principal:

Vizitator	**Sistem**
1. Cere înregistrare	
	2. Cere tipul utilizatorului
3. Selectează tipul utilizatorului	
	4. Afișează formularul corespunzător tipului selectat
5. Completează formularul și trimite datele [A1]	
	6. Verifică datele din punct de vedere sintactic și semantic [A2]
	7. Creează un cont nou
	8. Memorează datele contului
	9. Afișează profilul utilizatorului

Fluxuri alternative:

- ▷ [A1] Nu trimite datele sau renunță: utilizatorul anulează operația; sistemul reafișează pagina principală; fluxul se încheie.
- ▷ [A2] Date incorecte sau incomplete: sistemul afișează un mesaj de eroare și marchează câmpurile greșite; fluxul continuă cu pasul 4 din fluxul principal.

4.4.2 Cazul de utilizare: Autentificare

Nume: Autentificare
Descriere: Descrie comportamentul sistemului și interacțiunea cu utilizatorul pentru autentificare în sistem cu email și parolă.
Actor software: Utilizator
Eveniment declanșator: Utilizatorul cere autentificare.
Precondiții: Există cel puțin un cont înregistrat.
Postcondiții: Sistemul a inițializat sesiunea și a afișat opțiunile utilizatorului în funcție de rol.
Referințe încrucișate: Înregistrare utilizator

Flux principal:

Utilizator	**Sistem**
1. Cere să se autentifice	
	2. Cere email și parolă
3. Trimite datele cerute [A1]	
	4. Verifică datele introduse [A2]
	5. Creează sesiunea în funcție de utilizator
	6. Afișează opțiunile utilizatorului

Fluxuri alternative:

- [A1] Nu trimite datele sau renunță: utilizatorul anulează operația; sistemul reafișează pagina principală; fluxul se încheie.
- [A2] Date incorecte sau incomplete: sistemul afișează mesaj de eroare; fluxul continuă cu pasul 2 din fluxul principal.

4.4.3 Cazul de utilizare: Filtrare ateliere

Nume: Filtrare ateliere
Descriere: Afișează lista atelierelor și permite afișarea atelierelor după criterii, ca de exemplu categorie, preț, dată, interval vârstă.
Actor software: Vizitator sau utilizator
Eveniment declanșator: Actorul inițiază listarea atelierelor.
Precondiții: Sistemul a memorat date pentru cel puțin un atelier.
Postcondiții: Sistemul a afișat lista filtrată a atelierelor.
Referințe încrucișate: Vizualizare detalii atelier

Flux principal:

Vizitator/Utilizator	**Sistem**
1. Inițiază căutarea atelierelor	
	2. Afișează formularul de căutare (categorie, preț, dată)
3. Completează criteriile și trimite datele	
	4. Verifică datele introduse [A1]
	5. Caută atelierele pe baza criteriilor
	6. Afișează rezultatele căutării [A2]
7. Selectează un atelier pentru vizualizarea detaliilor	

Fluxuri alternative:

- ▷ [A1] Date incomplete sau greșite: sistemul afișează mesaj de eroare; fluxul continuă cu pasul 2 din fluxul principal.
- ▷ [A2] Nicio potrivire: sistemul afișează mesajul „Nu există ateliere potrivite”; fluxul continuă cu pasul 2 din fluxul principal.

4.4.4 Cazul de utilizare: Vizualizare detalii atelier

Nume: Vizualizare detalii atelier
Descriere: Descrie comportamentul sistemului și interacțiunea dintre utilizator și sistem pentru afișarea informațiilor complete ale unui atelier selectat (descriere, locație, instructor, recenzii).
Actor software: Vizitator sau utilizator
Eveniment declanșator: Actorul selectează un atelier din lista afișată.
Precondiții: Atelierul există și este activ sau vizibil.
Postcondiții: Sistemul a afișat pagina de detalii complete ale atelierului.
Referințe încrucișate: Vizualizare și filtrare ateliere. Înscriere la atelier.

Flux principal:

Vizitator/Utilizator	**Sistem**
1. Selectează atelierul din lista rezultată	
	2. Preia informațiile atelierului din baza de date [A1]
	3. Preia informațiile despre instructor și locație
	4. Preia recenziile asociate atelierului [A2]
	5. Calculează și afișează numărul locurilor ocupate
	6. Afișează pagina de detalii (descriere, dată, oră, preț, vârstă, instructor, locație, recenzii)

Fluxuri alternative:

- ▷ [A1] Atelier indisponibil: sistemul afișează mesaj „Atelier indisponibil”; revine la lista atelierelor.
- ▷ [A2] Nu există recenzii: sistemul afișează mesaj „Nu există recenzii pentru acest atelier”; fluxul continuă cu pasul 5.

4.4.5 Cazul de utilizare: Înscriere la atelier

Nume: Înscriere la atelier
Descriere: Descrie comportamentul sistemului și interacțiunea dintre participant și sistem în timpul înscrierii la un atelier.
Actor software: Participant
Eveniment declanșator: Participant selectează opțiunea de a se înscrie la un atelier.
Precondiții: Participant este autentificat. Atelierul este disponibil.
Postcondiții: Sistemul a creat o înregistrare asociată participantului și atelierului, data înscrierii este memorată și statusul înscrierii este setat implicit.
Referințe încrucișate: Autentificare. Anulare înscriere.

Flux principal:

Participant	**Sistem**
1. Selectează atelierul dorit	
	2. Afișează detaliile atelierului
3. Cere înscrierea la atelier	
	4. Verifică dacă utilizatorul este autentificat [A1]
	5. Verifică disponibilitatea atelierului [A2]
	6. Verifică dacă participantul are deja o înregistrare la acel atelier [A3]
	7. Creează înregistrarea (DataÎnscriere, Status)
	8. Memorează datele în baza de date
	9. Afișează mesaj de confirmare

Fluxuri alternative:

- ▷ [A1] Participant neautentificat: sistemul redirecționează către autentificare; după autentificare reușită, fluxul continuă cu pasul 2 din fluxul principal.
- ▷ [A2] Atelier indisponibil: sistemul afișează un mesaj de eroare; fluxul se încheie.
- ▷ [A3] Participant deja înscris: sistemul afișează un mesaj informativ; fluxul se încheie.

4.4.6 Cazul de utilizare: Anulare înscriere

Nume: Anulare înscriere
Descriere: Descrie comportamentul sistemului prin care un participant își anulează înscrierea la un atelier la care este deja înscris.
Actor software: Participant
Eveniment declanșator: Participantul selectează opțiunea de anulare a înscrierii.
Precondiții: Participantul este autentificat și există o înregistrare activă pentru atelier.
Postcondiții: Statusul înscrierii devine „Anulată”. Modificarea este memorată.
Referințe încrucișate: Înscriere la atelier

Flux principal:

Participant	Sistem
1. Accesează secțiunea de înscrieri	
	2. Afișează lista atelierelor la care participantul este înscris
3. Accesează atelierul dorit	
4. Selectează opțiunea de anulare a înscrierii	
	5. Solicită confirmarea operației [A1]
6. Confirmă anularea	
	7. Actualizează statusul înscrierii din „Activă” în „Anulată” [A2]
	8. Memorează modificarea
	9. Afișează mesaj de confirmare

Fluxuri alternative:

- ▷ [A1] Utilizatorul nu confirmă anularea: sistemul revine la lista înscrierilor; fluxul se încheie.
- ▷ [A2] Înscriere inexistentă: sistemul afișează mesaj de eroare; fluxul se încheie.

4.4.7 Cazul de utilizare: Gestionare atelier

Nume: Gestionare atelier
Descriere: Descrie comportamentul sistemului și interacțiunea dintre instructor și sistem în timpul creării sau modificării unui atelier.
Actor software: Instructor
Eveniment declanșator: Instructor cere adăugarea sau actualizarea atelierului.
Precondiții: Instructorul este autentificat.
Postcondiții: Atelierul este creat sau actualizat și memorat în sistem și în baza de date.
Referințe încrucișate: Autentificare. Asociere materiale

Flux principal:

Instructor	**Sistem**
1. Accesează secțiunea de gestionare a atelierelor	
	2. Afișează lista atelierelor create de instructor
3. Selectează una din opțiunile de creare sau modificare atelier	
	4. Afișează formularul corespunzător
5. Completează datele atelierului (denumire, descriere, temă, categorie, preț, interval vârstă, dată, oră, durată, locație)	
6. Trimite datele	
	7. Verifică datele din punct de vedere sintactic și semantic [A1]
	8. Verifică disponibilitatea datei, orei și locației [A2]
	9. Salvează datele
	10. Afișează mesaj de confirmare

Fluxuri alternative:

- ▷ [A1] Date invalide sau incomplete: sistemul marchează câmpurile incorecte; afișează mesaj de eroare; fluxul revine la pasul 4.
- ▷ [A2] Conflict cu alt atelier deja existent: sistemul afișează mesaj privind suprapunerea intervalului; fluxul revine la pasul 4.

4.4.8 Cazul de utilizare: Asociere materiale

Nume: Asociere materiale
Descriere: Descrie comportamentul sistemului și interacțiunea dintre instructor și sistem pentru asocierea materialelor necesare unui atelier și stabilirea cantităților utilizate.
Actor software: Instructor
Eveniment declanșator: Instructorul selectează opțiunea de gestionare a materialelor pentru un anumit atelier.
Precondiții: Instructor autentificat, atelier existent și aparținând lui. Materialele sunt definite în sistem.
Postcondiții: Sistemul a salvat utilizările, asociere atelier - material - cantitate. Sistemul poate calcula costul total al materialelor pentru atelier.
Referințe încrucișate: Gestionare atelier

Flux principal:

Instructor	**Sistem**
1. Accesează secțiunea de gestionare a atelierelor	
	2. Afișează lista atelierelor instructorului
3. Selectează atelierul pentru care dorește asocierea materialelor	
	4. Afișează detaliile atelierului și opțiunile acestuia
5. Selectează opțiunea de „Materiale”	
	6. Afișează lista materialelor disponibile și cantități
7. Selectează materialele și introduce cantitățile	
8. Trimite datele	
	9. Verifică validitatea cantităților (cantitate > 0) [A1]
	10. Verifică existența unei asocieri anterioare pentru același atelier și material [A2]
	11. Salvează asocierea atelier-material-cantitate
	12. Calculează costul total al materialelor
	13. Afișează mesaj de confirmare și costul estimat

Fluxuri alternative:

- ▷ [A1] Cantitate invalidă: sistemul afișează mesaj de eroare și marchează câmpurile incorecte; fluxul revine la pasul 6.
- ▷ [A2] Material deja asociat atelierului: sistemul afișează mesaj informativ; sistemul permite actualizarea cantității; fluxul revine la pasul 6.

4.4.9 Cazul de utilizare: Marcare participare efectivă

Nume: Marcare participare efectivă
Descriere: Descrie comportamentul sistemului și interacțiunea dintre instructor și sistem pentru marcarea participanților care au participat efectiv la atelier.
Actor software: Instructor
Eveniment declanșator: Instructorul finalizează atelierul și accesează lista participanților înscriși.
Precondiții: Instructorul este autentificat, atelierul este existent și există participanți înscriși.
Postcondiții: Statusul înscrierii este actualizat la „Participare” pentru participanții prezenți, numărul real de participanți este actualizat.
Referințe încrucișate: Anulare înscriere. Scriere recenzie

Flux principal:

Instructor	**Sistem**
1. Accesează secțiunea de gestionare a atelierelor	
	2. Afișează lista atelierelor instructorului
3. Selectează atelierul dorit	
	4. Sistemul verifică dacă atelierul este încheiat [A1]
5. Accesează lista participanților înscriși [A2]	
	6. Afișează lista participanților și statusul actual
7. Selectează participanții care au participat efectiv	
8. Confirmă operația	
	9. Actualizează statusul înscrierilor selectate la „Participare”
	10. Recalculează numărul real de participanți
	11. Memorează modificările
	12. Afișează mesaj de confirmare

Fluxuri alternative:

- ▷ [A1] Atelier neîncheiat: sistemul afișează un mesaj de eroare („Atelierul nu este încă finalizat”); fluxul se încheie.
- ▷ [A2] Nu există participanți înscriși: sistemul afișează mesaj informativ; fluxul se încheie.

4.4.10 Cazul de utilizare: Scriere recenzie

Nume: Scriere recenzie
Descriere: Descrie comportamentul sistemului și interacțiunea dintre participant și sistem pentru adăugarea unei recenzii aferente unui atelier la care a participat efectiv.
Actor software: Participant
Eveniment declanșator: Participantul selectează opțiunea de a adăuga o recenzie pentru un atelier finalizat.
Precondiții: Participantul este autentificat, există o înscriere cu Status=Participare și atelierul este încheiat.
Postcondiții: Recenzia este memorată în sistem, asociată înregistrării corespunzătoare și afișată pe pagina atelierului.
Referințe încrucișate: Marcare participare efectivă

Flux principal:

Participant	**Sistem**
1. Accesează secțiunea de înscrieri	
	2. Afișează lista atelierelor la care a participat
3. Selectează atelierul pentru care dorește să adauge recenzie	
	4. Verifică existența unei înregistrări cu status „Participare” [A1]
	5. Verifică dacă atelierul este încheiat [A2]
	6. Verifică dacă există deja o recenzie pentru această înregistrare [A3]
	7. Afișează formularul de recenzie (rating, comentariu)
8. Introduce ratingul și comentariul	
9. Trimite datele	
	10. Verifică validitatea ratingului (1-5) [A4]
	11. Salvează recenzia
	12. Afișează mesaj de confirmare

Fluxuri alternative:

- ▷ [A1] Participant neeligibil: sistemul afișează mesaj („Nu puteți adăuga recenzie fără participare efectivă la atelier.”); fluxul se încheie.
- ▷ [A2] Atelier neîncheiat: sistemul afișează mesaj („Recenzia poate fi adăugată doar după finalizarea atelierului”); fluxul se încheie.
- ▷ [A3] Recenzie deja existentă: sistemul afișează un mesaj informativ; fluxul se încheie.
- ▷ [A4] Rating invalid: sistemul afișează un mesaj de eroare; fluxul revine la pasul 7.

Capitolul 5

Modulul de Inteligență Artificială

5.1 Introducere

În cadrul aplicației au fost integrate două module de inteligență artificială, acestea având ca scop generarea automată de descrieri pentru atelierele creative și rezumarea recenziilor primite de către instructori la atelierele susținute. Funcționalitățile au fost implementate cu ajutorul modelului de limbaj de mari dimensiuni (LLM) Llama 3, rulat local prin intermediul platformei Ollama [**?**].

Ambele module utilizează modelul de limbaj Llama 3, dezvoltat de Meta AI, rulat local prin platforma Ollama. Ollama permite descărcarea și rularea modelelor LLM pe propriul sistem, fără a necesita o conexiune la un serviciu extern sau o cheie API. Comunicarea cu modelul se realizează prin intermediul unui API REST local.

5.2 Generarea automată a descrierilor pentru workshop-uri

5.2.1 Scopul funcționalității

Funcționalitatea permite generarea automată a unei descrieri pentru un atelier, pe baza informațiilor completate de organizator. Sistemul poate genera o descriere generală, obiectivele workshop-ului și un text privind publicul țintă. Aceasta are rolul de a ajuta organizatorii să completeze mai rapid informațiile despre ateliere și să obțină o descriere pe care să o folosească în întregime, să o corecteze și să o folosească sau să le ofere o viziune asupra a ce să scrie.

5.2.2 Fluxul funcționalității

Promptul transmis modelului pentru generarea automată a descrierilor este construit dinamic, pe baza datelor completate de instructor în formularul de creare a atelierului. Acesta include: titlul, categoria, tema, nivelul de dificultate, durata și intervalul de vârstă recomandat.

Structura promptului cuprinde trei secțiuni distincte: datele de intrare, cerințele de conținut și cerințele de format. Cerințele de conținut specifică lungimea minimă a descrierii (80 de cuvinte), numărul exact de obiective de învățare și formatul textului despre publicul țintă. Cerințele de format impun returnarea unui obiect JSON valid, cu câmpurile description, objectives și targetText.

Procesul de generare se desfășoară în următoarele etape:

1. Organizatorul completează formularul workshop-ului.
2. Datele sunt trimise către backend.
3. Backend-ul construiește automat promptul.
4. Promptul este transmis către modelul LLM prin Ollama.
5. Modelul generează răspunsul în format JSON.
6. Sistemul validează automat structura și conținutul răspunsului.
7. În cazul unui răspuns invalid, sistemul reîncearcă automat generarea (maximum 3 încercări), transmițând modelului și motivele eșecului.
8. Rezultatul valid este afișat utilizatorului și poate fi salvat sau editat în funcție de preferințele acestuia și de calitatea generării.

5.2.3 Evaluarea rezultatelor

Evaluarea funcționalității de generare automată a descrierilor a fost realizată utilizând un set de exemple generate pe baza unor workshop-uri fictive. Aceasta a fost realizată semi-automat, combinând verificări automate ale structurii cu evaluarea manuală a calității textului generat.

Validarea automată include procese precum: verificarea formatului JSON, verificarea existenței câmpurilor obligatorii, verificarea tipului de date, verificarea lungimii minime a descrierii, verificarea numărului exact de 3 obiective generate, verificarea structurii obiectivelor, verificarea ca fiecare obiectiv să nu depășească 12 cuvinte, verificarea ca textul despre publicul țintă să aibă 1-2 propoziții și detectarea frazelor stilistice interzise.

Validarea manuală urmărește: relevanța rezultatului față de datele de intrare, claritatea și coerența textului, corectitudinea gramaticală și lingvistică, precum și necesitatea intervenției umane.

Fiecare criteriu a fost evaluat pe o scară de la 1 la 5, iar scorul final a fost calculat utilizând formula:

$$\text{scor_final} = \frac{\text{format_ok} + \frac{\text{relevanța}}{5} + \frac{\text{claritatea}}{5} + \frac{\text{corectitudinea}}{5}}{4}$$

Interpretarea scorurilor a fost realizată astfel: peste 0,85 este un rezultat foarte bun, între 0,7 și 0,85 un rezultat bun, între 0,5 și 0,7 un rezultat mediu, iar sub 0,5 un rezultat slab.

Medii per criteriu:

Criteriu	**Medie**
Relevanță	3,12 / 5
Claritate	3,83 / 5
Corectitudine lingvistică	3,60 / 5

Distribuție scoruri:

Interval	**Interpretare**	**Cazuri**
$> 0{,}85$	Foarte bun	2
$0{,}7 - 0{,}85$	Bun	38
$0{,}5 - 0{,}7$	Mediu	7
$< 0{,}5$	Slab	3

Privind rezultatele, modelul se descurcă rezonabil. Media scorului final este **0,778**, ceea ce îl plasează în categoria „bun". Practic, în 94% din cazuri, textele generate au o calitate satisfăcătoare pentru utilizare practică, cu necesitatea unor revizuiri ocazionale. Există totuși câteva situații în care modelul eșuează complet sau care l-au dus sub pragul mediu. Un aspect pozitiv este consistența, modelul neavând variații mari.



Figura 5.1: Exemplu de descriere generată automat pentru un atelier



```
{
  "title": "Workshop bijuterii handmade",
  "input": {
    "category": "DIY",
    "level": "Avansat",
    "duration": "4 ore",
    "target_audience": "Copii",
    "activity_type": "Practic",
    "material": "margele si ata"
  },
  "generated": {
    "description": "În cadrul workshopului de bijuterii handmade, participanții vor învăța să creeze obiecte unice și personalizate folosind margele și ata. Această activitate practică este destinată copiilor și are ca scop dezvoltarea creativității și a abilităților manuale. În timp de 4 ore, partici
    "objectives": [
      "a invata tehnici de lucru cu margele si ata",
      "a exersa abilitatea de a selecta materialele potrivite",
      "a realiza o bijuterie handmade personalizată"
    ],
    "targetText": "Copii"
  },
  "valid": true,
  "format_ok": 1,
  "attempt_count": 3,
  "failure_reasons": [],
  "attempts": [
    {
      "attempt_number": 1,
      "raw_response": "{\n  \"description\": \"Acest workshop de bijuterii handmade este destinat copiilor care doresc să învețe tehnici de creație a bijuteriei. În cadrul acestui eveniment, participanții vor învăța cum să confecționeze bijuterii unice și personale folosind margele și ata. Ei vor exer
      "parsed_response": {
        "description": "Acest workshop de bijuterii handmade este destinat copiilor care doresc să învețe tehnici de creație a bijuteriei. În cadrul acestui eveniment, participanții vor învăța cum să confecționeze bijuterii unice și personale folosind margele și ata. Ei vor exersa abilitatea de a
        "objectives": [
          "Să înveți tehnica de confecționare a bijuteriei handmade",
          "Să exersezi abilitatea de a crea forme și structuri complexe",
          "Să aplici cunoștințele și abilitățile învățate pentru a crea bijuterii personale"
        ],
        "targetText": "Copiii între 8 și 12 ani"
      },
      "valid": false,
      "failure_reasons": [
        "too_short_description",
        "objective_not_infinitive"
      ]
    },
    {
      "attempt_number": 2,
      "raw_response": "{\n  \"description\": \"În cadrul workshopului de bijuterii handmade, participanții vor învăța să creeze piese unice și personalizate folosind margele și ata. Această activitate practică este destinată copiilor și are ca scop dezvoltarea creativității și a abilităților de a lucr
      "parsed_response": {
        "description": "În cadrul workshopului de bijuterii handmade, participanții vor învăța să creeze piese unice și personalizate folosind margele și ata. Această activitate practică este destinată copiilor și are ca scop dezvoltarea creativității și a abilităților de a lucra cu materiale diferi
        "objectives": [
          "a învăța tehnici de asamblare și decorare a bijuteriilor",
          "a exersa selectarea și prepararea margelelor și a atelor pentru crearea bijuteriilor",
          "a realiza o lucrare simplă, cum ar fi un inel sau o brățară"
        ],
        "targetText": "Copii"
      },
      "valid": false,
      "failure_reasons": [
        "objective_too_long"
      ]
    },
    {
      "attempt_number": 3,
      "raw_response": "{\n  \"description\": \"În cadrul workshopului de bijuterii handmade, participanții vor învăța să creeze obiecte unice și personalizate folosind margele și ata. Această activitate practică este destinată copiilor și are ca scop dezvoltarea creativității și a abilităților manua
      "parsed_response": {
        "description": "În cadrul workshopului de bijuterii handmade, participanții vor învăța să creeze obiecte unice și personalizate folosind margele și ata. Această activitate practică este destinată copiilor și are ca scop dezvoltarea creativității și a abilităților manuale. În timp de 4 ore,
        "objectives": [
          "a invata tehnici de lucru cu margele si ata",
          "a exersa abilitatea de a selecta materialele potrivite",
          "a realiza o bijuterie handmade personalizată"
        ],
        "targetText": "Copii"
      },
      "valid": true,
      "failure_reasons": []
    }
  ]
}
```


Figura 5.2: Exemplu de răspuns JSON cu mecanism de reîncercare (3 încercări)

5.2.4 Limitările modulului de generare a descrierilor

Deși modulul produce rezultate satisfăcătoare în majoritatea cazurilor, există limitări inerente ale utilizării unui model de limbaj în acest context:

1. Calitatea descrierii depinde de datele introduse de instructor. Dacă titlul, categoria sau durata sunt incomplete sau prea generale, descrierea generată poate fi vagă sau repetitivă.
2. Modelul poate produce formulări prea generale, insuficient adaptate specificului atelierului.
3. Descrierea generată trebuie validată de instructor înainte de publicare, pentru a evita informații incorecte sau promisiuni nerealiste.
4. Rezultatele pot varia pentru intrări similare, ceea ce poate afecta consistența platformei.
5. Modulul oferă o descriere inițială, dar nu poate surprinde întotdeauna particularitățile pedagogice sau organizatorice ale atelierului.

5.3 Rezumarea automată a feedback-ului pentru workshop-uri

5.3.1 Scopul funcționalității

Funcționalitatea permite generarea automată a unui rezumat al recenziei primite de instructori de la participanți după un workshop. Pe baza recenziilor oferite de participanți, sistemul generează patru secțiuni: un rezumat general al părerilor participanților, punctele forte, punctele slabe și sugestiile de îmbunătățire. Această funcționalitate are rolul de a ajuta organizatorii să interpreteze mai ușor părerile participanților fără a fi nevoie să parcurgă fiecare recenzie.

5.3.2 Fluxul funcționalității

Procesul de rezumare se desfășoară în următoarele etape:

1. Sistemul colectează recenziile unui workshop din baza de date.
2. Backend-ul colectează toate comentariile text (non-vide) ale tuturor recenziilor existente pentru acel atelier, printr-o interogare JOIN între tabelele recenzie și inregistrare.
3. Comentariile sunt numerotate și concatenate într-o listă, apoi incluse simultan într-un singur prompt transmis modelului. Nu există o limită explicită de tokeni implementată, toate comentariile disponibile sunt trimise într-un singur apel.
4. Modelul generează răspunsul într-un text structurat, cu etichete predefinite.
5. Sistemul validează automat structura răspunsului. Acesta trebuie să regăsească prezența tuturor secțiunilor, lungimea minimă a rezumatului general și cel puțin un element de tip bullet.
6. În cazul unui răspuns invalid, sistemul reîncearcă automat generarea, transmițând modelului și motivele eșecului.
7. Rezultatul valid este afișat organizatorului și poate fi salvat sau revizuit.

5.3.3 Evaluarea rezultatelor

Evaluarea funcționalității a fost realizată pe un set de 10 workshop-uri, fiecare cu câte 10 recenzii. Evaluarea a fost realizată semi-automat, combinând verificări automate cu evaluare manuală a calității textului.

Validarea automată include: verificarea prezenței tuturor secțiunilor, verificarea lungimii minime a rezumatului (îi cerem modelului mai mult, mai exact 50 de cuvinte, dar acceptăm și mai puțin, 30 de cuvinte), verificarea existenței cel puțin a unui element de tip bullet în secțiunile de puncte forte, puncte slabe și sugestii.

Validarea manuală urmărește acoperirea ideilor principale, fidelitatea față de recenzii, corectitudinea gramaticală, fluența și plasarea corectă a informațiilor în secțiuni. Fiecare criteriu a fost evaluat pe o scară de la 1 la 5.

Scorul final a fost calculat utilizând formula:

$$\text{scor_final} = \frac{\text{format_ok} + \frac{\text{acoperire}}{5} + \frac{\text{fidelitate}}{5} + \frac{\text{claritate}}{5} + \frac{\text{categorisire}}{5}}{5}$$

Interpretarea scorurilor a fost realizată astfel: peste 0,85 este un rezultat foarte bun, între 0,7 și 0,85 un rezultat bun, între 0,5 și 0,7 un rezultat mediu, iar sub 0,5 un rezultat slab.

Medii per criteriu:

Criteriu	**Medie**
Acoperire idei principale	4,00 / 5
Fidelitate față de recenzii	3,90 / 5
Claritate formulări	3,50 / 5
Corectitudine categorizare	4,40 / 5

Distribuție scoruri:

Interval	**Interpretare**	**Cazuri**
$> 0{,}85$	Foarte bun	5
$0{,}7 - 0{,}85$	Bun	3
$0{,}5 - 0{,}7$	Mediu	2
$< 0{,}5$	Slab	0

Conform rezultatelor, modelul se comportă bine în rezumarea feedback-ului. Media scorului final este 0,832 și este în categoria „bun". Practic, în 100% din cazuri, textele generate au depășit pragul minim de calitate. Un aspect pozitiv este rata de succes la prima generare aproape pentru toate rezumatele. Nu a existat niciun eșec complet.

```
[1/10] Workshop ID 1: Atelier pictura acrilica
  Se genereaza rezumat: Atelier pictura acrilica (10 recenzii, rating mediu: 4.2/5)
    - Incercarea 1/3
    Raspuns valid.
```

Figura 5.3: Exemplu de rezumat al feedback-ului generat automat



Figura 5.4: Exemplu de răspuns JSON generat de modulul de analiză a recenziilor

5.3.4 Limitările modulului de rezumare a feedback-ului

1. Calitatea rezumatului depinde de numărul și relevanța recenziilor. Dacă există puține comentarii sau acestea sunt scurte, rezumatul poate fi superficial.
2. Modelul poate omite opinii minoritare, feedback-ul dominant poate fi favorizat.
3. Comentariile ironice, foarte scurte sau ambigue pot fi interpretate incorect.
4. Rezumatul poate simplifica excesiv informația.
5. Rezumatul trebuie considerat un instrument de asistență în redactare pentru instructor, nu o interpretare a tuturor opiniilor utilizatorilor.

Capitolul 6

Prezentarea aplicației

Aplicația CraftManager este structurată în două componente: un backend FastAPI și un frontend React, care comunică prin API REST. Backend-ul este organizat în routere independente (`auth`, `ateliere`, `inregistrari`, `recenzii`, `generate`, `admin`, `materiale`), fiecare responsabil de câte o funcționalitate. Frontend-ul folosește un `AuthContext` global pentru gestionarea sesiunii și un serviciu Axios centralizat care atașează automat tokenul JWT la fiecare cerere HTTP.

Capitolul prezintă funcționalitățile din perspectiva utilizatorului, urmărind parcursul fiecărui tip de utilizator, fie vizitator neautentificatserviciu Axios, fie participant, instructor sau administrator.

6.1 Pagina principală

Pagina principală este accesibilă oricărui tip de utilizator și vizitatorilor, prezintă o secțiune hero cu o bară de căutare rapidă, urmată de o selecție de ateliere disponibile cu un buton care te duce la secțiunea de ateliere. Navbar-ul se adaptează în funcție de rolul utilizatorului autentificat: participanții văd *Înscrierile mele*, instructorii văd *Dashboard*, iar administratorul *Administrare*.

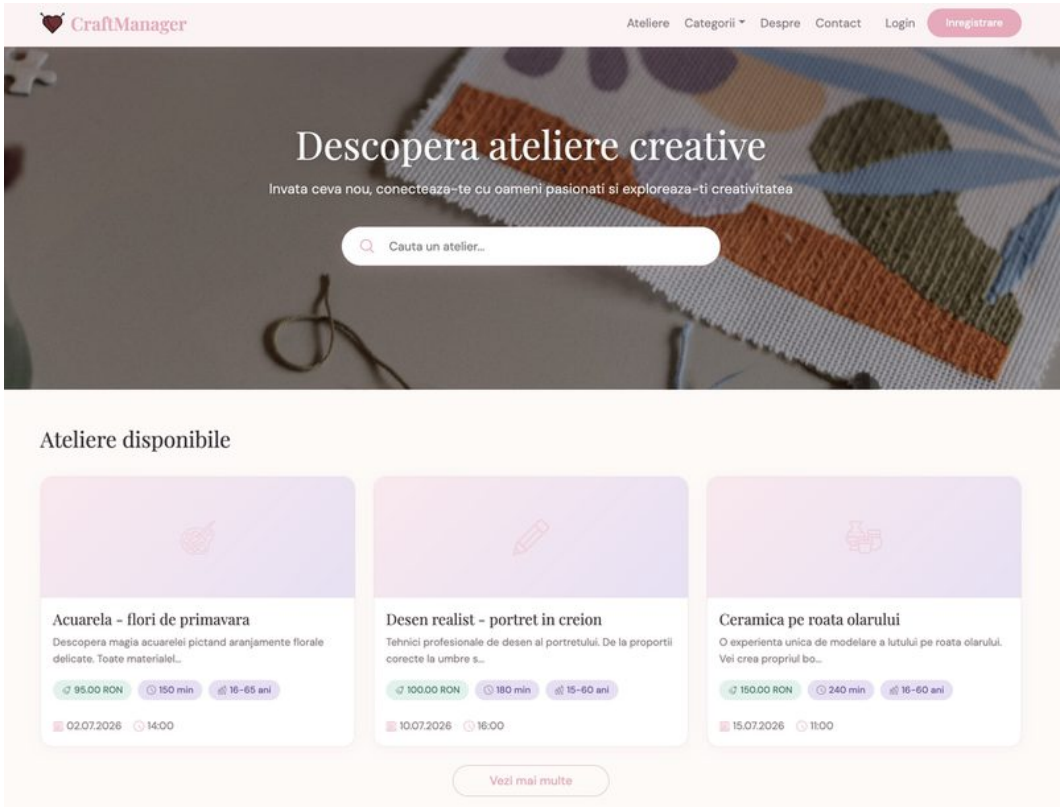


Figura 6.1: Pagina principală

6.2 Înregistrarea și autentificarea utilizatorilor

La înregistrare, utilizatorul completează datele personale și alege tipul de cont: participant sau instructor. Formularul este validat înainte de trimitere, folosind expresii regulate care acceptă caractere Unicode pentru a suporta diacriticele:

Listing 6.1: Validare client

```
// Accepta orice litera din orice limba (inclusiv diacritice)
if (!/^[\p{L}\s-]+$/u.test(form.nume.trim()))
  return 'Numele poate contine doar litere.'
```

```
// Verificare unicitate email (pe server)
if (db.query(Utilizator).filter(
      Utilizator.email == data.email).first():
  raise HTTPException(400, "Email deja inregistrat")
```

Dacă adresa de email este deja înregistrată, serverul returnează HTTP 400, iar frontend-ul afișează mesajul de eroare în formular.

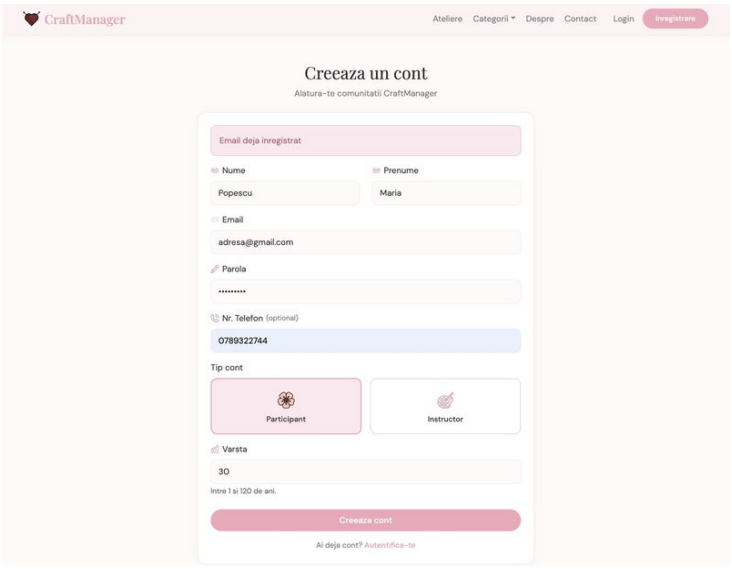


Figura 6.2: Formular de înregistrare cu mesaj de eroare pentru email deja înregistrat

La autentificare, serverul generează un token JWT stocat în `localStorage`. Contextul React `AuthContext` preia tokenul la fiecare reîncărcare a paginii, menținând sesiunea fără o nouă autentificare.

6.3 Secțiunea de ateliere

Pagina `/ateliere` afișează toate atelierele înregistrate care urmează să aibă loc. Avem filtrare pe categorii prin butoane rapide și filtru avansat care permite combinarea mai multor criterii: preț, interval de date, oraș, vârstă și nivel. Pe backend, query-ul SQLAlchemy este dinamic, sunt aplicate doar filtre pentru care utilizatorul a furnizat valori. Avem și condiții ca valorile minime să nu fie mai mari decât cele maxime.

Listing 6.2: Filtrare dinamică a atelierelor — GET /ateliere

```
query = query.filter(
  text("atelier.data + atelier.ora > NOW()"))  # doar viitoare
if pret_min: query = query.filter(Atelier.pret >= pret_min)
if pret_max: query = query.filter(Atelier.pret <= pret_max)
if varsta:   query = query.filter(
```

```
                Atelier.varstamin <= varsta,
                Atelier.varstamax >= varsta)
if categorie: query = query.join(Tema).filter(
                Tema.idcategorie == categorie)
```

Condiția `data + ora > NOW()` exclude automat atelierele deja desfășurate.

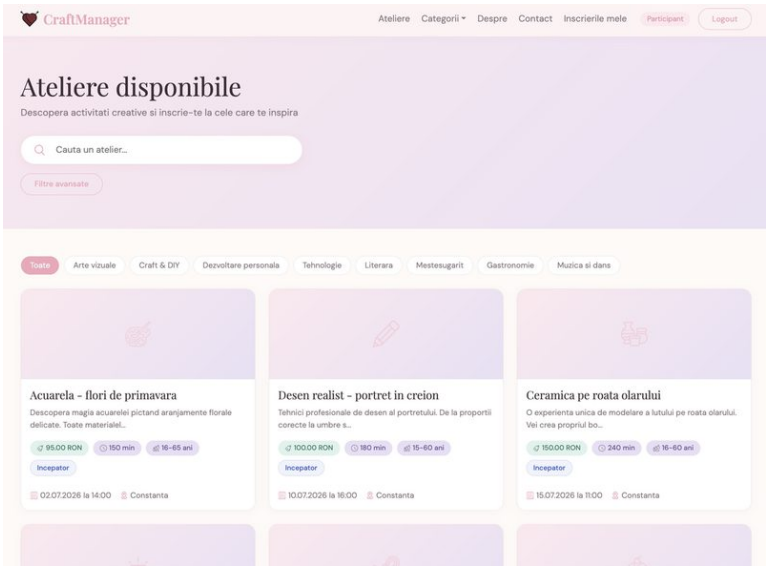


Figura 6.3: Secțiunea de ateliere cu filtrare pe categorii

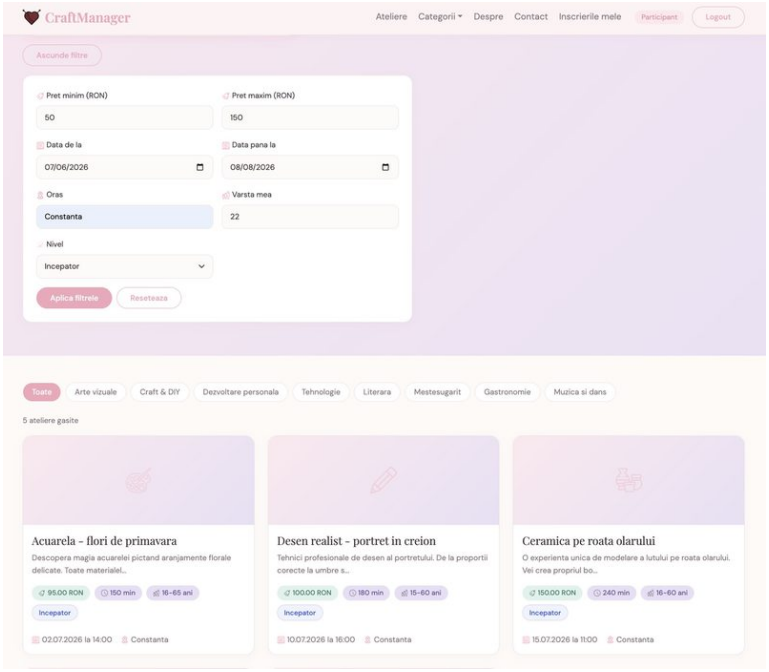


Figura 6.4: Filtre avansate: preț, dată, oraș, vârstă, nivel

6.4 Înscrierile participantului

Pagina `/inscrierile-mele` afișează atelierele la care participantul s-a înscris, delimitate în *Înscrieri active* și *Ateliere trecute*. Pentru atelierele încheiate cu status *Participare* oferit de instructor, participantul poate adăuga o recenzie cu rating de la 1 la 5 stele și un comentariu opțional. Butonul *Recenzie* devine *Recenzie trimisă* după trimitere, prevenind abuzul recenziilor.

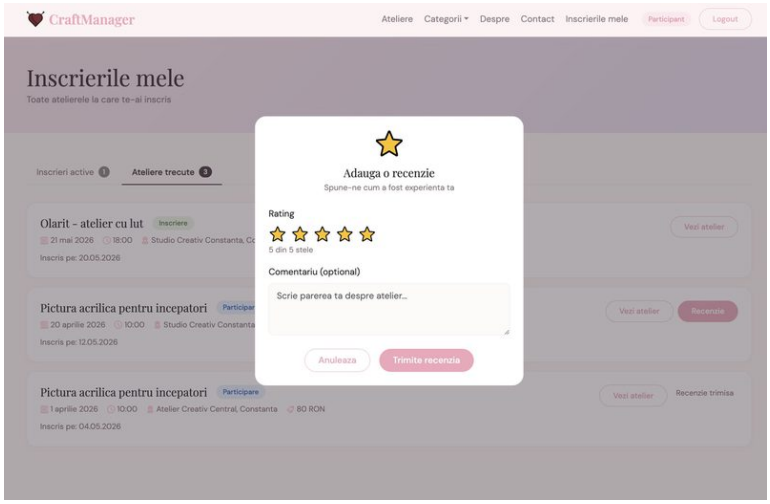


Figura 6.5: Modalul de adăugare recenzie pentru un atelier încheiat

6.5 Dashboard-ul instructorului

6.5.1 Crearea atelierelor și generarea de descrieri cu ajutorul AI

Instructorul poate crea ateliere noi completând formularul cu toate informațiile necesare. O funcționalitate care vine în ajutorul instructorilor este generarea automată a descrierii: la apăsarea butonului *Generează descriere*, datele din formular sunt trimise modulului AI. Răspunsul este generat în cateva secunde, constând în descriere, obiective și public țintă. Acesta este afișat, iar instructorul îl poate accepta sau ignora. Daca îl accepta, acesta este adaugat în secțiune și poate fi îmbunătățit, schimbat sau șters complet.

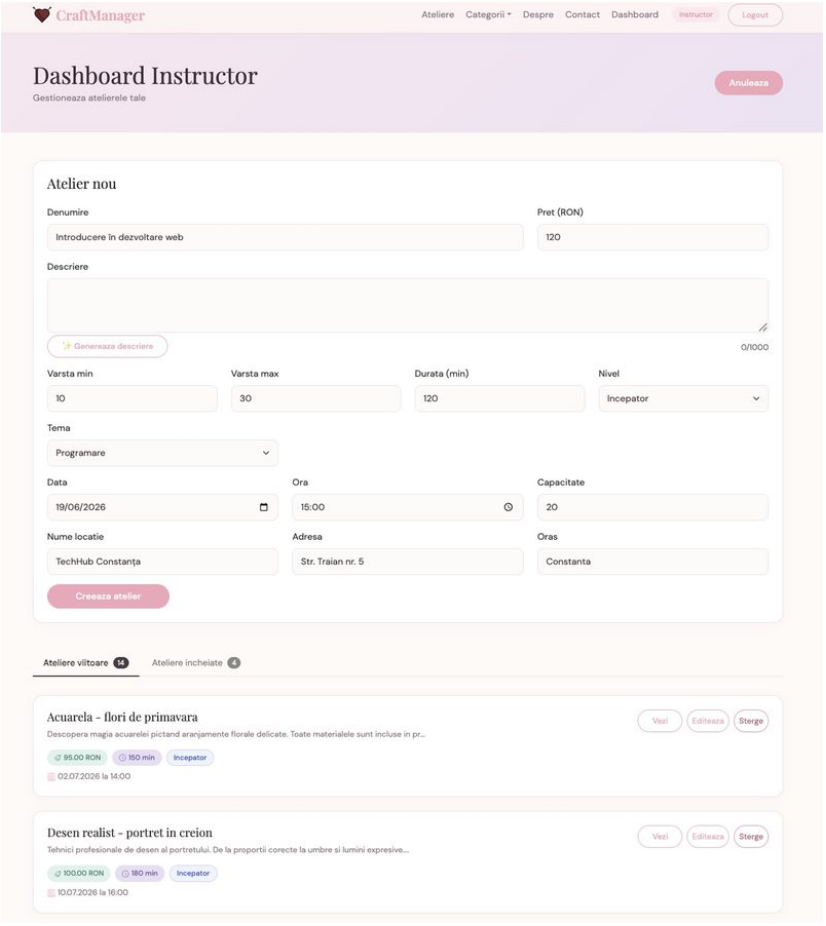


Figura 6.6: Formular de creare atelier în dashboard-ul instructorului

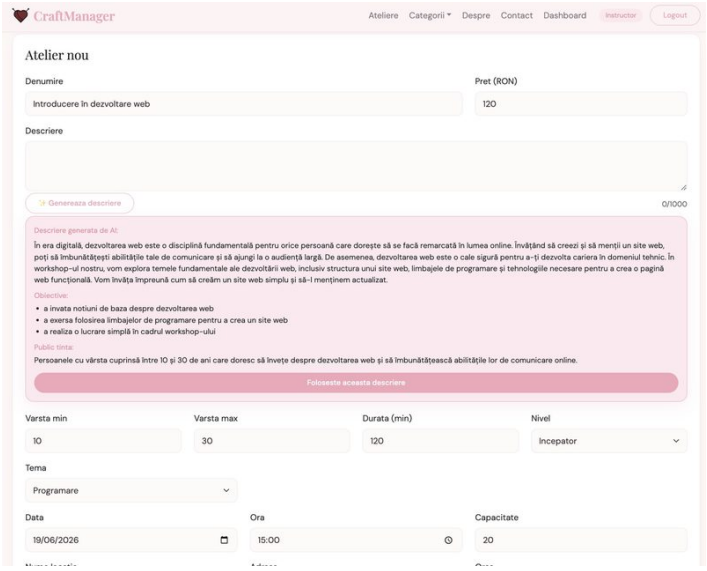


Figura 6.7: Descriere generată de modulul AI

6.5.2 Evaluările primite și rezumatul AI

Pentru fiecare atelier încheiat, instructorul poate vedea recenziile primite de la participanți, cu rating și comentariu, precum și media generală a stelelor. Butonul *Regenerează rezumat* trimite toate recenziile modulului AI. Acesta produce automat un rezumat în patru secțiuni: rezumat general, puncte forte, puncte slabe și sugestii de îmbunătățire.

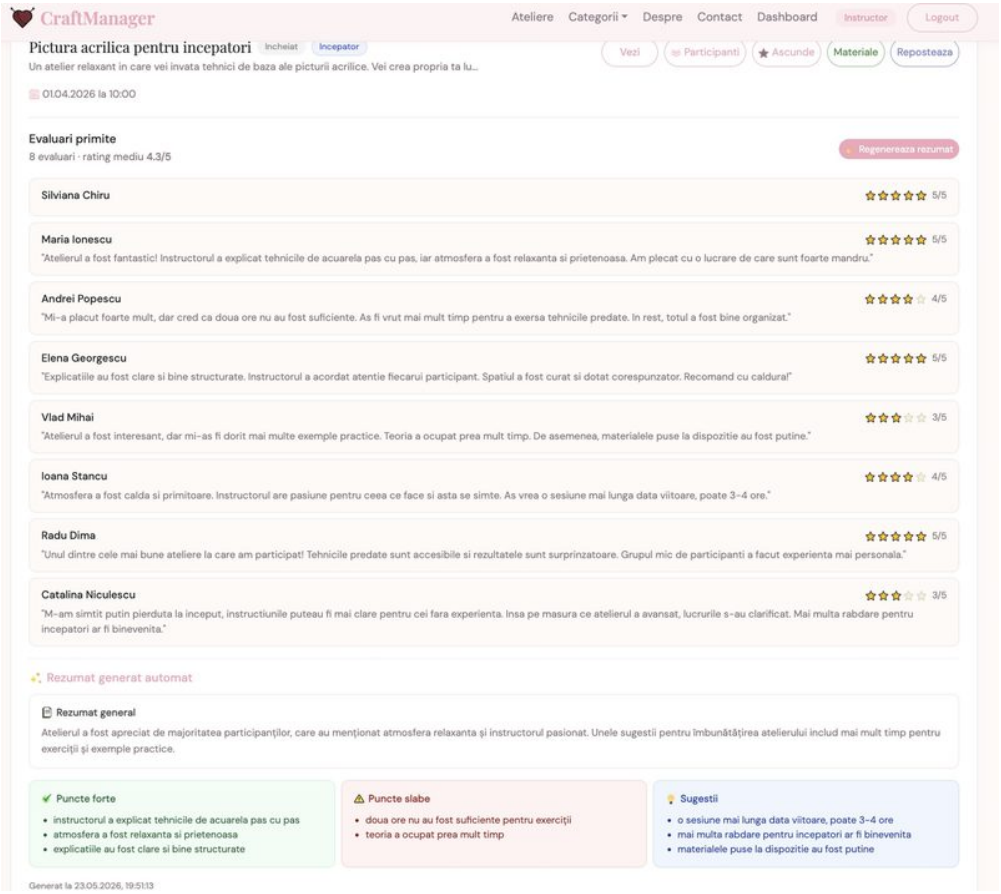


Figura 6.8: Evaluările primite și rezumatul generat de AI

6.5.3 Raportul financiar

Pagina `/contul-meu` a instructorului include și un raport financiar care calculează pentru fiecare atelier încheiat veniturile raportate la numărul de participanți cu status *Participare* care este înmulțit cu prețul atelierului, cheltuielile pe materiale și profitul net. Participanții care au anulat înscrierea sau nu au participat efectiv la atelier nu sunt incluși în calcul.

Listing 6.3: Calculul profitului per atelier

```
castigat = count(Inregistrare WHERE status='Participare')
           * atelier.pret
cheltuit = sum(Utilizare.cantitate * Materiale.pret)
profit   = castigat - cheltuit
```

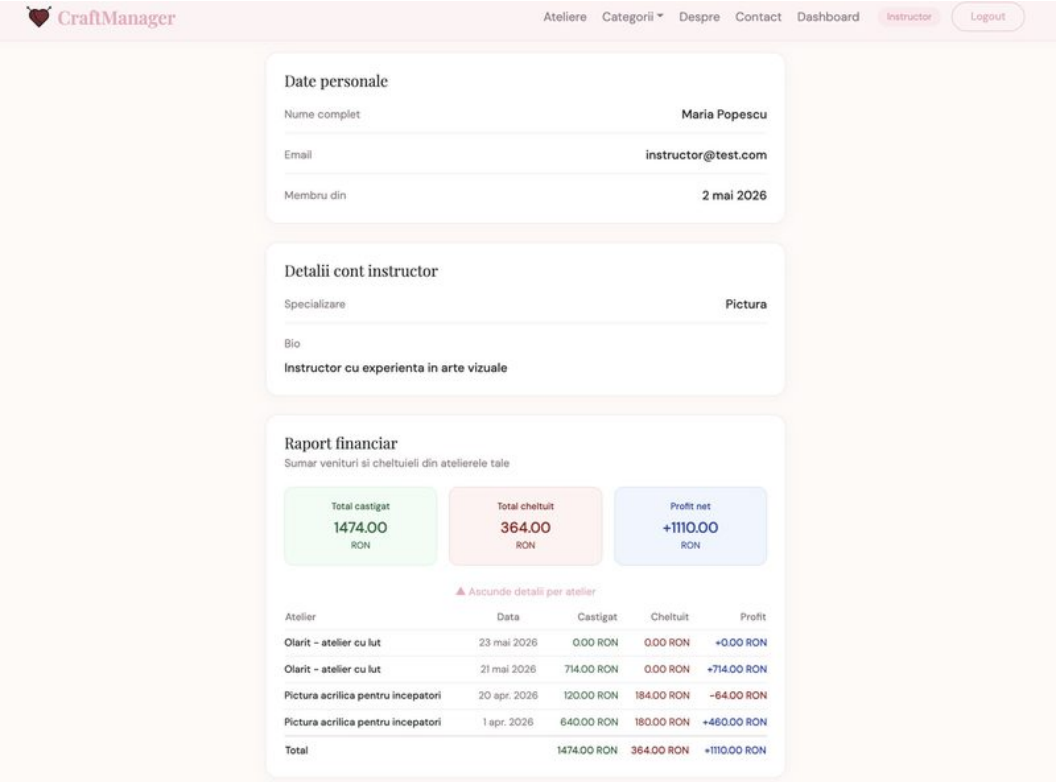


Figura 6.9: Raportul financiar cu detalii per atelier

6.6 Panoul de administrare

Administratorul poate gestiona platforma dintr-un panou cu patru secțiuni: *Categorii*, *Mesaje*, *Utilizatori* și *Ateliere*. În secțiunea Categorii se definesc categoriile și temele asociate, având posibilitatea și să șteargă teme sau categorii întregi. Secțiunea Utilizatori afișează toți utilizatorii cu tipul contului și data înregistrării, cu posibilitatea de ștergere. Mesajele din formularul de contact se regăsesc în secțiunea Mesaje.

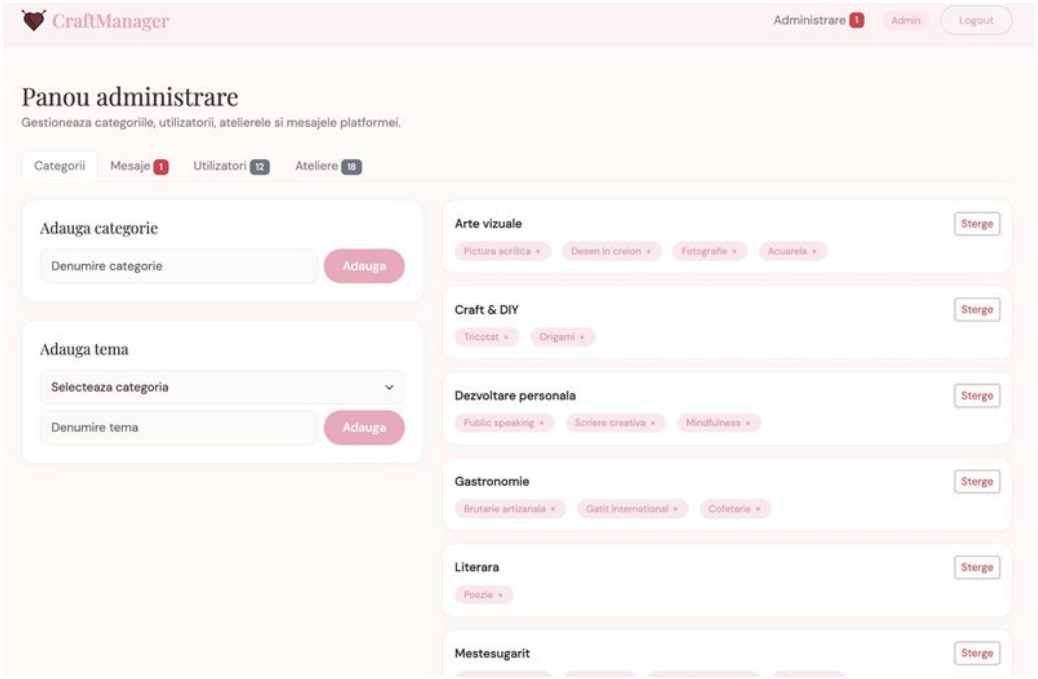


Figura 6.10: Panoul de administrare, secțiunea teme și categorii

6.7 Securitatea aplicației

Autentificarea se bazează pe JWT: tokenul generat la login este atașat automat la fiecare cerere HTTP prin interceptori Axios. În backend, dependența `get_utilizator_curent()` validează tokenul, iar `get_admin_curent()` verifică suplimentar dacă rolul utilizatorului este de administrator. Parolele sunt stocate exclusiv ca hash `bcrypt`. Accesul la resurse este restricționat pe roluri, instructorii modificând doar propriile ateliere, participanții adaugând recenzii doar la atelierele la care au participat efectiv, iar acțiunile administrative fiind disponibile exclusiv administratorilor.

Capitolul 7

Testarea aplicației

Capitolul prezintă procesul de testare al aplicației pe care l-am realizat odată manual, prin interacțiunea directă cu interfața și odată cu Postman, prin apelarea directă a endpoint-urilor backend-ului.

7.1 Testare funcțională manuală

Testarea manuală a urmărit verificarea funcționalităților principale ale aplicației prin introducerea de date și observarea comportamentului aplicației. Pentru fiecare test s-au notat datele introduse și rezultatul obținut.

Tabelul 7.1: Rezultatele testării funcționale manuale

Funcționalitate	**Status**
Autentificare	
Înregistrare participant	Trecut
Email duplicat la înregistrare	Trecut
Autentificare cu date corecte	Trecut
Autentificare cu parolă greșită	Trecut
Ateliere și înscrieri	
Filtrare după categorie	Trecut
Filtrare după preț	Trecut
Înscriere la atelier	Trecut
Înscriere duplicată	Trecut
Adăugare recenzie fără participare efectivă	Trecut
Dashboard instructor	
Creare atelier	Trecut
Generare descriere AI	Trecut
Raport financiar	Trecut
Administrare	
Acces panou admin fără cont administrator	Trecut
Ștergere utilizator	Trecut
Navbar adaptat după rolul utilizatorului	Trecut

Toate cele 15 teste funcționale au fost trecute. Aplicația gestionează corect atât fluxurile principale, cât și cazurile limită, date invalide, acces neautorizat și operațiuni duplicate.

7.2 Testare API cu Postman

Testarea API a fost realizată cu Postman, prin apelarea directă a endpoint-urilor backend-ului FastAPI. Au fost testate cinci scenarii, verificând atât comportamentul la date valide, cât și răspunsurile la cereri neautorizate sau resurse inexistente.

7.2.1 Înregistrare utilizator - POST /auth/register

Primul test verifică crearea unui cont nou de participant. Cererea trimite datele utilizatorului în format JSON, iar serverul returnează codul `201 Created` și mesajul de confirmare.

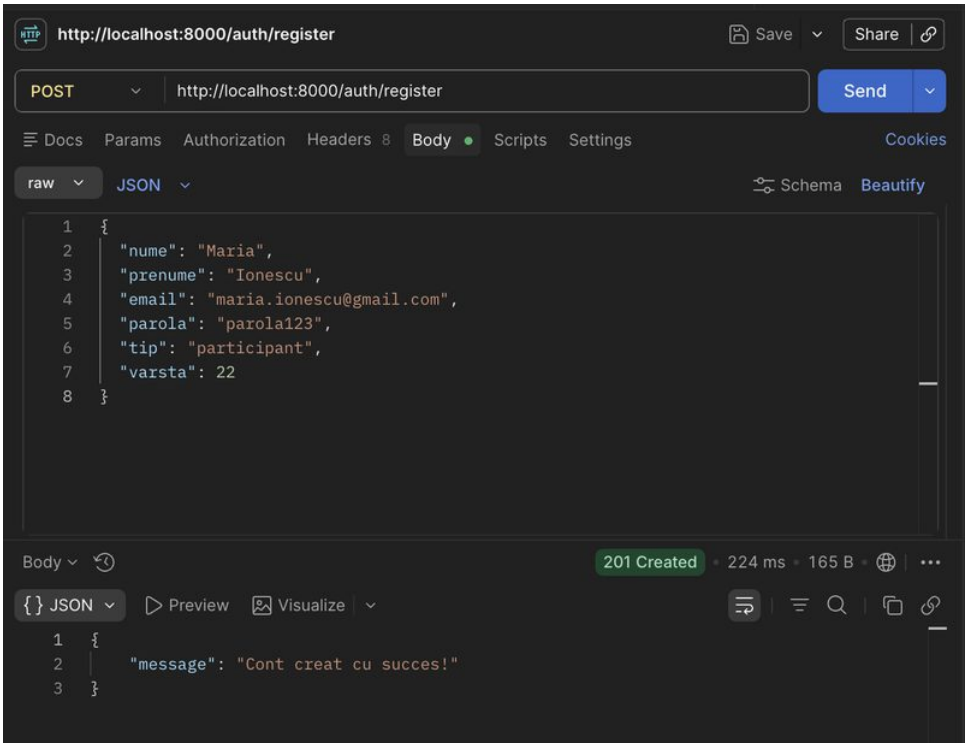


Figura 7.1: Postman - POST /auth/register, răspuns 201 Created

7.2.2 Autentificare - POST /auth/login

Al doilea test verifică autentificarea cu credențiale valide. Serverul returnează un token JWT care va fi folosit pentru autorizarea cererilor ulterioare.

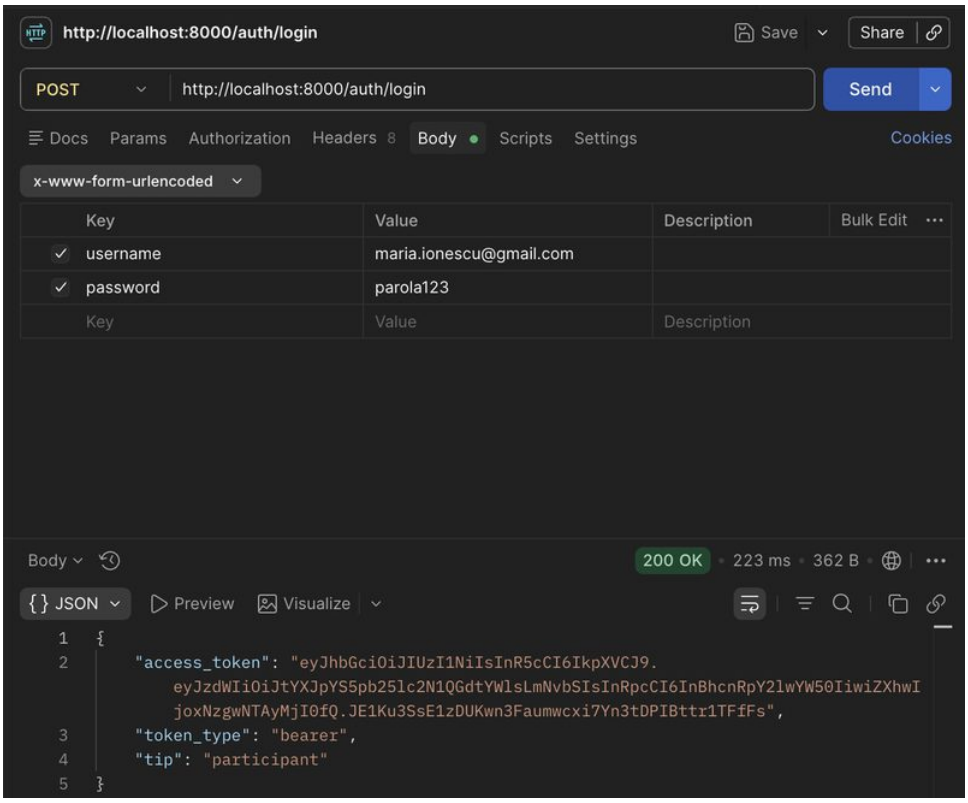


Figura 7.2: Postman - POST /auth/login, răspuns 200 OK cu token JWT

7.2.3 Catalog ateliere - GET /ateliere/

Al treilea test verifică accesul public la lista de ateliere, fără autentificare. Serverul returnează `200 OK` și lista completă a atelierelor disponibile.

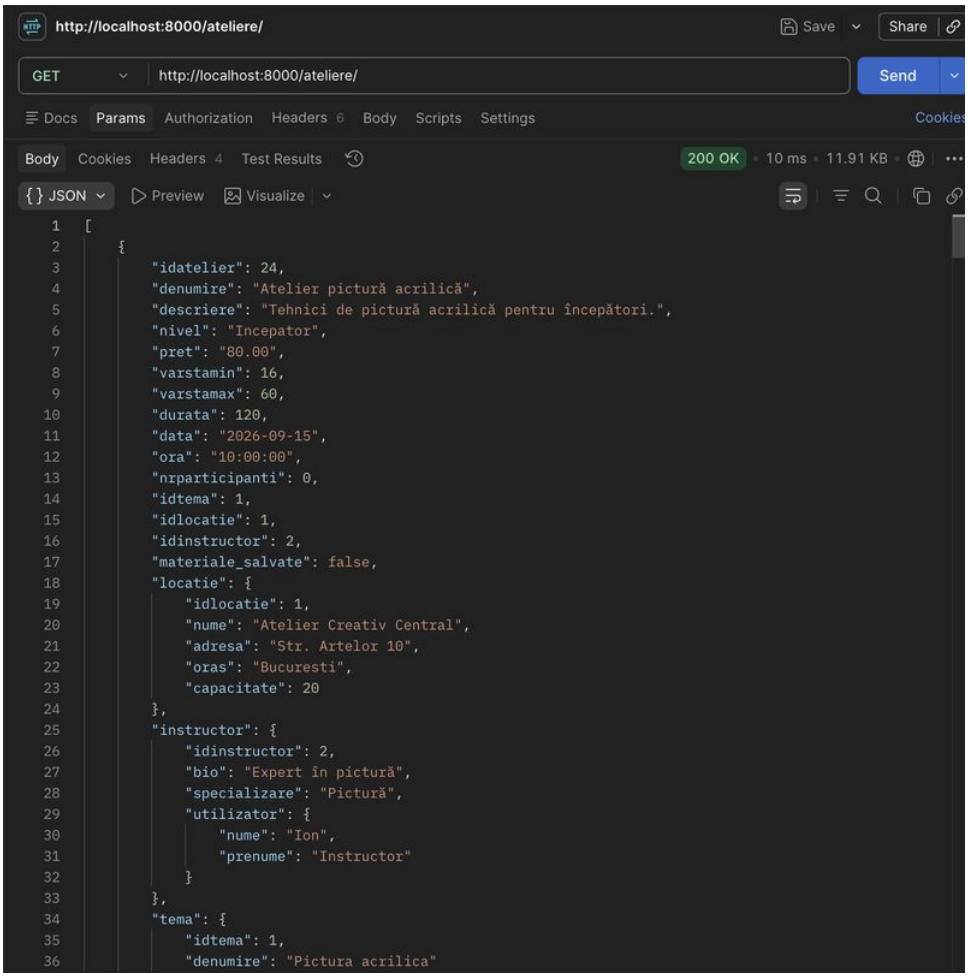


Figura 7.3: Postman - GET /ateliere/, răspuns 200 OK fără autentificare

7.2.4 Înscriere la atelier - POST /inregistrari/

Al patrulea test verifică înscrierea unui participant autentificat la un atelier existent. Tokenul JWT este transmis în header-ul `Authorization`. Serverul returnează `201 Created` și confirmarea înscrierii.

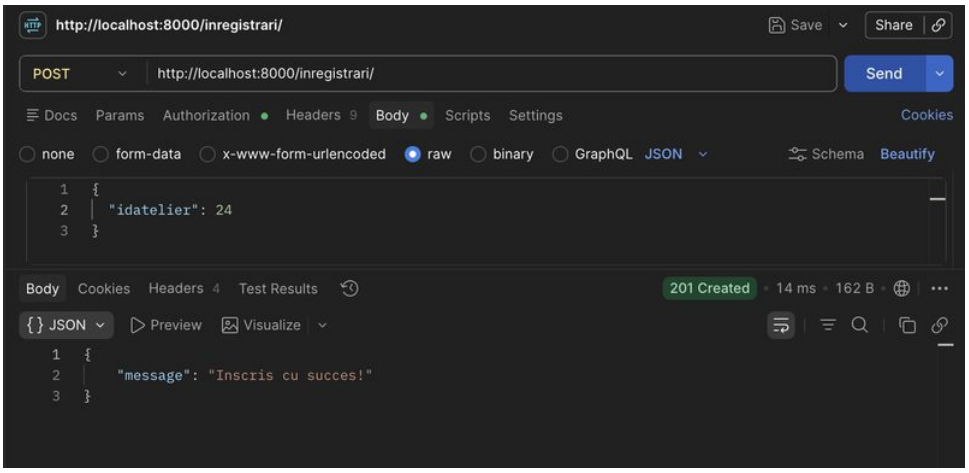


Figura 7.4: Postman - POST /inregistrari/ cu token valid, răspuns 201 Created

7.2.5 Acces neautorizat - POST /inregistrari/ fără token

Ultimul test verifică protecția endpoint-ului de înscriere. Fără token de autentificare, serverul returnează `401 Unauthorized` și mesajul „Not authenticated", arătând că accesul la resurse protejate este bine restricționat.

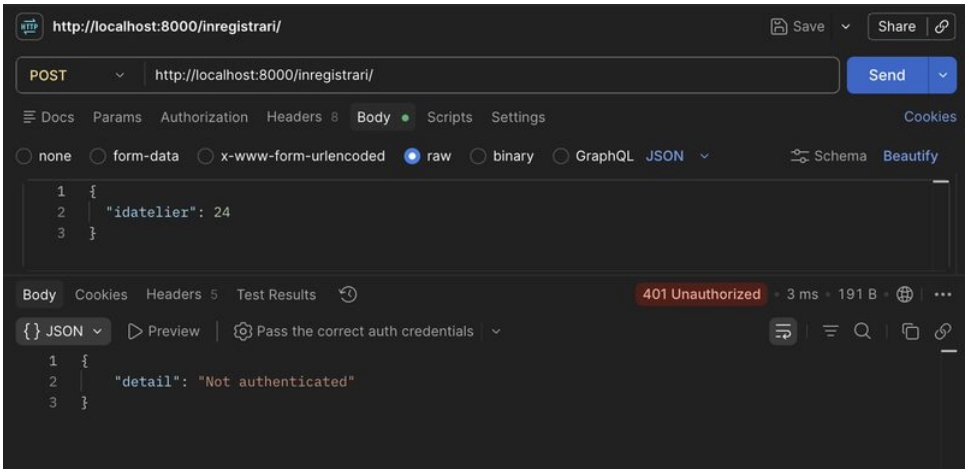


Figura 7.5: Postman - POST /inregistrari/ fără token, răspuns 401 Unauthorized

Tabelul 7.2: Rezumatul testelor API realizate în Postman

Endpoint	**Metodă**	**Date trimise**	**Răspuns obținut**	**Status**
/auth/register	POST	Date utilizator JSON	201 + „Cont creat cu succes!"	Trecut
/auth/login	POST	username + password	200 + token JWT	Trecut
/ateliere/	GET	—	200 + lista ateliere	Trecut
/inregistrari/	POST	idatelier + token	201 + „Inscris cu succes!"	Trecut
/inregistrari/	POST	idatelier, fără token	401 Unauthorized	Trecut

Cele 5 teste API au aratat că backend-ul se comportă corect. Endpoint-urile publice sunt accesibile fără autentificare, cele protejate resping cererile neautorizate și datele valide sunt procesate și salvate corect.

Capitolul 8

Concluzii și direcții viitoare

8.1 Concluzii

8.2 Dificultăți întâmpinate

8.3 Direcții viitoare